

Computational Deep Learning

Theory, Algorithms, and Frontiers

Computational Deep Learning

Theory, Algorithms, and Frontiers

A Three-Volume Monograph
Covering Mathematical Foundations,
Algorithms & Architectures,
and Research Frontiers

MAHESH SOLANKI
scitamehtam research group

Computational Deep Learning: Theory, Algorithms, and Frontiers

Copyright © 2025 Mahesh Solanki.

All rights reserved.

No part of this publication may be reproduced, stored in a retrieval system, or transmitted in any form or by any means—electronic, mechanical, photocopying, recording, or otherwise—without the prior written permission of the author, except in the case of brief quotations embodied in critical reviews and certain other noncommercial uses permitted by copyright law.

This monograph is licensed under a Creative Commons Attribution-NonCommercial-ShareAlike 4.0 International License (CC BY-NC-SA 4.0). Source code listings within this work are licensed under the MIT License.

Independently published by the author.

First edition: 2025.

Author: Mahesh Solanki

Research group: scitamehtam research group

Typeset in Linux Libertine and DejaVu Sans Mono.

Printed in black and white on demand.

THE AUTHOR AND PUBLISHER HAVE MADE EVERY EFFORT TO ENSURE THE ACCURACY OF THE INFORMATION HEREIN. HOWEVER, THE WORK IS PROVIDED “AS IS” WITHOUT WARRANTY OF ANY KIND.

*For every reader who refuses to take
the algorithm on faith.*

Contents

Preface	8	5 Information Theory	34
0.1 Why this monograph, and why now	9	5.1 Entropy and mutual information	36
0.2 What “computational” means in the title	9	5.2 The data processing inequality	36
0.3 The plan of the work	9	5.3 The information bottleneck principle	36
0.4 A note on rigor	10	5.4 Variational inference and the ELBO	37
0.5 A note on the computational material	10	5.5 Cross-entropy and softmax	37
0.6 How this book was written	10	5.6 Computational experiments	37
Welcome	10	5.7 Historical notes	38
0.7 What you will find	12	5.8 Exercises	38
0.8 How to read this monograph	12		
0.9 Conventions	12	II. Algorithms and Architectures	39
0.10 Acknowledgments	12		
I. Mathematical Foundations	14	6 Introduction to Volume II	40
1 Introduction to Volume I	15	6.1 Chapter map	41
1.1 Why these four	16	6.2 How to read	41
1.2 Chapter map	16		
1.3 How to read	16	7 Multilayer Perceptrons	41
2 Linear Algebra for Neural Computation	17	7.1 Universal approximation	43
2.1 Vector spaces, norms, and inner products	19	7.2 Activation functions	43
2.2 Spectral theory	19	7.3 Initialization	43
2.3 The singular value decomposition	20	7.4 Normalization	44
2.4 The geometry of high-dimensional spaces	21	7.5 Dropout	44
2.5 Tensors and tensor decompositions	22	7.6 A complete MLP in PyTorch	45
2.6 Structured linear operators in deep learning	22	7.7 Computational experiments	45
2.7 Computational experiments	23	7.8 Historical notes	45
2.8 Historical notes	23	7.9 Exercises	45
2.9 Exercises	24	8 Convolutional Networks	46
3 Optimization	24	8.1 Translation equivariance	48
3.1 Convex optimization: the complete theory	26	8.2 The convolution operation	48
3.2 Stochastic gradient descent	27	8.3 Canonical architectures	48
3.3 The loss surface of deep networks	27	8.4 Pooling and invariance	49
3.4 Adaptive optimizers	28	8.5 Receptive field	49
3.5 Computational experiments	29	8.6 Computational experiments	49
3.6 Historical notes	29	8.7 Historical notes	49
3.7 Exercises	30	8.8 Exercises	49
4 Probability and Concentration	30	9 Recurrent Networks	50
4.1 Basic inequalities	32	9.1 The vanishing and exploding gradient problem	52
4.2 Sub-Gaussian and sub-exponential random variables	32	9.2 LSTM and GRU	52
4.3 McDiarmid’s inequality	33	9.3 Connection to dynamical systems	52
4.4 Uniform convergence and classical learning theory	33	9.4 Computational experiments	53
4.5 Computational experiments	34	9.5 Historical notes	53
4.6 Historical notes	34	9.6 Exercises	53
4.7 Exercises	34	10 The Transformer	53
		10.1 Self-attention	55
		10.2 Positional encodings	55
		10.3 The transformer block	55
		10.4 Encoder, decoder, encoder-decoder	56
		10.5 Efficient attention	56
		10.6 Computational experiments	56
		10.7 Historical notes	57
		10.8 Exercises	57

11 Generative Models	57	16.8 Computational experiments	78
11.1 Variational autoencoders	59	16.9 Historical notes	79
11.2 Generative adversarial networks	59	16.10 Exercises	79
11.3 Diffusion models	59	17 Interpretability	79
11.4 Computational experiments	60	17.1 Feature visualization	81
11.5 Historical notes	60	17.2 Circuits	81
11.6 Exercises	60	17.3 Sparse autoencoders for interpretability	81
12 The Practice of Training	60	17.4 Causal interventions	81
12.1 Initialization revisited	62	17.5 The microscope vs. MRI distinction	82
12.2 Optimizer selection	62	17.6 Computational experiments	82
12.3 Normalization revisited	62	17.7 Historical notes	82
12.4 Regularization	62	17.8 Exercises	82
12.5 Distributed training	62	18 Open Problems	83
12.6 Monitoring training	63	18.1 Optimization	85
12.7 Computational experiments	63	18.2 Generalization	85
12.8 Historical notes	63	18.3 Representation	86
12.9 Exercises	63	18.4 Architectures	86
III. Frontiers and Open Problems	65	18.5 Interpretability	87
13 Introduction to Volume III	66	18.6 Training and data	87
13.1 Chapter map	67	18.7 Theory and practice	87
13.2 How to read	67	18.8 Coda	87
14 Representation Learning	67	References	89
14.1 The manifold hypothesis	69	9 Foundational deep learning	90
14.2 Linear representations	69	10 Transformers and attention	90
14.3 Superposition and polysemantic neurons	69	11 Optimization	90
14.4 Sparse autoencoders	70	12 Architectures	90
14.5 Probing and causal interventions	70	13 Generative models	90
14.6 Computational experiments	70	14 Generalization theory	90
14.7 Historical notes	70	15 Scaling laws	90
14.8 Exercises	70	16 Interpretability	91
15 Scaling Laws	71	17 Mathematical foundations	91
15.1 The Kaplan scaling law	73	18 Information theory in deep learning	91
15.2 The Chinchilla correction	73	19 Training practice	91
15.3 Scaling beyond Chinchilla	73	20 Efficient attention	91
15.4 Theoretical explanations	74		
15.5 Implications for compute-optimal training	74		
15.6 Computational experiments	74		
15.7 Historical notes	74		
15.8 Exercises	74		
16 Generalization Theory	75		
16.1 The classical theory is vacuous	77		
16.2 Implicit bias of SGD	77		
16.3 Flat minima and PAC-Bayes	77		
16.4 Compression-based bounds	78		
16.5 The neural tangent kernel	78		
16.6 The lottery ticket hypothesis	78		
16.7 The state of the theory	78		

FRONT MATTER

Preface

Deep learning has, in less than two decades, moved from a marginal subfield of machine learning — sustained by a small community of persistent researchers — to a dominant paradigm in artificial intelligence, with measurable impact on computer vision, speech recognition, natural language processing, scientific computing, and increasingly on the empirical sciences themselves. The defining methodological fact of this transformation is deceptively simple: very large parametric function classes, trained by stochastic gradient descent on very large datasets, can generalize remarkably well. The defining theoretical puzzle is that they do so in regimes where classical learning theory predicts they should not.

0.1 Why this monograph, and why now

The existing literature on deep learning falls roughly into three categories: (i) course-oriented textbooks such as [goodfellow2016deep] and [bishop2006pattern], which are pedagogical and broad; (ii) survey articles on individual topics, which are deep but narrow; and (iii) the primary research literature, which is current but fragmented across conferences and preprint servers. A monograph that integrates the three — taking the time to develop the mathematics honestly, to implement the algorithms explicitly, and to situate both against the empirical and theoretical frontier — is overdue.

This book is written for the reader who has moved beyond introductions and is ready for the harder material: convergence proofs for Adam, the neural tangent kernel, double descent, mechanistic interpretability, scaling laws. The reader who can already train a transformer but wants to understand *why* a transformer. The reader who has read the original papers but wants them integrated into a single coherent story.

0.2 What “computational” means in the title

We use the word *computational* deliberately. Three senses are intended:

1. **Computational in the algorithmic sense.** Every result in this book is accompanied by an explicit algorithm — pseudocode in the chapter, executable code in the companion notebooks. We are unwilling to leave “the algorithm” as folklore.
2. **Computational in the resource sense.** Modern deep learning is shaped as much by the geometry of GPU memory, the bandwidth of interconnect, and the cost of communication as by any mathematical consideration. Where these constraints matter, we discuss them.
3. **Computational in the experimental sense.** The field progresses by experiments, and theory that ignores the empirical record is of limited use. Each chapter reports key empirical findings — loss curves, scaling plots, ablations — with full provenance.

0.3 The plan of the work

Volume I develops the four pillars on which the rest of the book stands. Linear algebra is presented not as a refresher but as the language of neural computation: tensor factorizations, spectral methods, low-rank approximation, randomized numerical linear algebra, and the geometry of high-dimensional spaces. Optimization is developed in two passes: convex optimization (where the theory is essentially complete, following [boyd2004convex] and [nesterov2018lectures]) and non-convex optimization (where the theory is partial and the empirical record matters). Probability and concentration inequalities are developed to the depth required to read the generalization literature. Information theory is developed to the depth required to read both the representation learning literature and the variational inference literature.

Volume II treats the principal architectures of modern deep learning in a uniform style: each is introduced as a function class with a specific inductive bias, the relevant theorems (expressivity, universal approximation, equivariance) are proved or cited, the training algorithm is given explicitly, and the empirical performance is reported. The volume closes with a chapter on the practice of training — initialization, normalization, regularization, optimization, and the engineering of large-scale training runs.

Volume III turns to the open questions. Why do overparameterized networks generalize? What does the geometry of the loss surface look like, and why

does it matter? What do scaling laws tell us about the structure of the learning problem? How should we understand the representations learned by deep networks, and can we read them? The volume closes with an explicit list of open problems — twenty of them — that we believe will shape the next decade.

0.4 A note on rigor

We have chosen rigor where rigor is illuminating and citation where it is not. Proofs are given in full when the proof technique itself is illuminating (the Cybenko–Hornik universal approximation proof, the NTK derivation, the Adam convergence argument under decreasing steps). Proofs are sketched or cited when the full argument is technical but the conclusion is widely used (the Johnson–Lindenstrauss lemma, the manifold tangent approximation, the Bartlett–Mendelson Rademacher bound). We have made every effort to ensure that every cited result is correctly attributed; errors should be reported to the repository’s issue tracker.

0.5 A note on the computational material

All code in the monograph is written in Python, using PyTorch as the primary deep learning framework and NumPy / SciPy / Matplotlib for scientific computing and plotting. The companion notebooks in the `notebooks/` directory of the repository are executable: each can be run end-to-end on a single modern GPU in under an hour, and most can be run on a laptop CPU in under ten minutes. The repository includes a `Makefile` and a `pyproject.toml` specifying the exact environment; CI runs all notebooks on every commit.

0.6 How this book was written

This monograph is itself a computational artifact. The source is a Quarto project — markdown files with embedded code, rendered to HTML, PDF (via Tectonic), and EPUB from a single source of truth. The repository is public, version-controlled, and continuously built. Readers are invited to file issues, suggest improvements, and submit pull requests.

— **scitamehtam research group** · Mahesh Solanki
Summer, 2025

FRONT MATTER

Welcome

This monograph is a self-contained, three-volume treatment of **computational deep learning** — the mathematical, algorithmic, and empirical study of neural networks trained by gradient-based optimization on large datasets. The aim is to provide a researcher-grade reference that moves fluidly between rigorous theory, implementable algorithms, reproducible computational experiments, and a candid map of the open problems that define the field’s frontier.

The work is organized around three convictions that have shaped the field since [lecun2015deep]:

1. **Deep learning is a computational discipline.** Theorems matter, but so does the cost of matrix multiplications, the structure of GPU memory hierarchies, the curvature of the loss surface, and the empirical scaling behavior of training loss. Theory and computation are inseparable.
2. **Architectures are best understood as constraints on the function class, not as recipes.** Convolution, attention, recurrence, and residual connections are each a different inductive bias. A monograph should clarify what each buys and what each costs.
3. **The frontier moves quickly, but the questions are stable.** Specific architectures, optimizers, and benchmarks rise and fall; the underlying questions about generalization, optimization, representation, and interpretability persist. A monograph should orient the reader toward the questions.

0.7 What you will find

The three volumes proceed from foundations to frontier:

- **Volume I — Mathematical Foundations.** Linear algebra for neural computation, convex and non-convex optimization, probability and concentration, and information theory. Each chapter ends with explicit notes on how the material surfaces in deep learning.
- **Volume II — Algorithms and Architectures.** Multilayer perceptrons, convolutional networks, recurrent networks, the transformer family, generative models (VAEs, GANs, diffusion), and the prac-

tice of training (initialization, normalization, optimization, regularization).

- **Volume III — Frontiers and Open Problems.** Representation learning and the geometry of feature spaces, scaling laws and compute-optimal training, the theory of generalization in overparameterized regimes, interpretability and mechanistic understanding, and a chapter devoted to explicit open problems.

Every chapter contains: formal definitions and theorems with proofs where the proof is instructive; pseudocode algorithm boxes; runnable Python listings (companion notebooks in the notebooks/ directory); computational experiments with explicit numerical results; exercises graded by difficulty; historical notes; and a bibliography.

0.8 How to read this monograph

The intended reader has the mathematical maturity of a beginning PhD student in computer science, statistics, physics, or applied mathematics, and is comfortable with the basics of probability, linear algebra, and multivariate calculus at the level of [strang2016introduction] and [bishop2006pattern]. Familiarity with Python is assumed for the computational sections; familiarity with PyTorch is helpful but not required.

Volumes are designed to be read in any order, although Volume I is a prerequisite for the proofs in Volumes II and III. The computational notebooks can be read alongside the chapters or run independently; each is self-contained.

0.9 Conventions

We use the following notation throughout:

Throughout, theorems, lemmas, propositions, and corollaries are numbered consecutively within each chapter. Algorithms, definitions, examples, and remarks share the same counter. Exercises are numbered separately at the end of each chapter.

0.10 Acknowledgments

This monograph draws on a vast body of work by the deep learning community. References to the primary literature are given at the end of each chapter

Symbol	Meaning
$\mathbb{R}^d$	The d -dimensional real vector space
$\mathbf{x} \in \mathbb{R}^d$	A column vector; components x_i
$\mathbf{X} \in \mathbb{R}^{m \times n}$	A real matrix; entries X_{ij}
$\mathbf{A}^\top, \mathbf{A}^{-1}$	Transpose and inverse
$\ \mathbf{x}\ _p$	The ℓ_p norm; $\ \mathbf{x}\ $ defaults to ℓ_2
$\nabla f, \nabla^2 f$	Gradient and Hessian of f
$\mathbb{E}[X], \text{Var}(X)$	Expectation and variance of a random variable
$\mathcal{N}(\mu, \Sigma)$	Gaussian distribution with mean μ , covariance Σ
$D_{\text{KL}}(P\ Q)$	Kullback–Leibler divergence
$H(X)$	Shannon entropy
f_θ	A parametric function with parameters θ
$\mathcal{L}(\theta)$	A loss function; $\hat{\mathcal{L}}$ is the empirical loss

and in the consolidated bibliography. We are grateful to the many colleagues whose questions, corrections, and discussions have shaped the presentation.

— scitamehtam research group · Mahesh Solanki, 2025

Part I.

Mathematical Foundations

Introduction to Volume I

Volume I develops the four pillars of mathematics on which the rest of the monograph rests: **linear algebra**, **optimization**, **probability**, and **information theory**. Each is presented at the depth required to read current research in deep learning theory, with explicit attention to the points where the mathematics surfaces in the algorithms and proofs of Volumes II and III.

1.1 Why these four

A deep learning system is, mathematically, four things at once. It is a **parametric function class** — a family of maps $f_\theta : \mathcal{X} \rightarrow \mathcal{Y}$ indexed by parameters $\theta \in \mathbb{R}^P$. The geometry of this function class, and of the parameter space that indexes it, is linear-algebraic and differential-geometric. It is an **optimization problem** — find the parameter θ^* that minimizes an empirical loss. The theory of how this optimization proceeds, why it converges, and what it converges to is the theory of stochastic gradient descent and its variants. It is a **probabilistic model** — the data are drawn from an unknown distribution, the network’s predictions are interpreted as probabilities, and generalization is a statement about the gap between empirical and population risk. Finally, it is an **information-processing system** — the network transforms inputs through a sequence of representations, and the information-theoretic properties of these representations (mutual information with the input, with the label, with each other) are increasingly central to the theory of representation learning.

The four chapters of this volume treat these four aspects in turn. The treatment is *not* a survey: it is a development of the specific results, tools, and proof techniques that recur in the deep learning literature.

1.2 Chapter map

Chapter 1 — Linear Algebra for Neural Computation treats the spectral theory of matrices, the SVD and low-rank approximation, randomized numerical linear algebra, the geometry of high-dimensional spaces (concentration of measure, near-orthogonality, the Johnson–Lindenstrauss lemma), tensors and tensor decompositions, and the structured linear operators (convolution, attention) that arise in neural network architectures.

Chapter 2 — Optimization treats convex optimization (gradient descent, accelerated methods, the convergence theory of [nesterov2018lectures]) and non-convex optimization (SGD, the Polyak–Łojasiewicz condition, the role of saddle points, the loss surface geometry of [choromanska2015loss]). The chapter closes with a treatment of the adaptive optimizers (AdaGrad, RMSProp, Adam, AdamW) that dominate modern practice, and a careful statement of the convergence results that are known — and the gaps that remain.

Chapter 3 — Probability and Concentration treats the basic inequalities (Markov, Chebyshev, Chernoff), sub-Gaussian and sub-exponential random variables, the Bernstein–Hoeffding bound, McDiarmid’s inequality, and the uniform convergence bounds (VC dimension, Rademacher complexity) that underlie classical learning theory. The chapter pays particular attention to the regimes where classical bounds are *vacuous* for deep networks — the entry point for the modern generalization theory of Volume III.

Chapter 4 — Information Theory treats entropy, mutual information, the data processing inequality, the chain rules, the asymptotic equipartition property, and the information bottleneck principle. The chapter closes with the role of information-theoretic quantities in variational inference (ELBO, the evidence lower bound) and in the information bottleneck view of deep learning due to [tishby2015deep].

1.3 How to read

The four chapters are largely independent and can be read in any order, with two caveats. Chapter 2 (Optimization) assumes the spectral theory of Chapter 1, specifically the operator norm and the SVD. Chapter 4 (Information Theory) assumes the basic probability of Chapter 3, specifically concentration inequalities. A reader interested only in the optimization theory of deep learning can read Chapters 1 and 2 and skip to Volume II. A reader interested only in the generalization theory should read Chapters 1, 3, and 4 before Volume III.

The computational experiments at the end of each chapter are designed to be runnable on a laptop in under ten minutes. The exercises range from routine (verifying definitions) to research-level (open

questions suitable for a thesis). Difficulty is marked as Easy / Medium / Hard / Research.

Linear Algebra for Neural Computation

The computational primitives of deep learning are linear-algebraic: matrix multiplication, tensor contraction, low-rank factorization, projection onto subspaces. Even the name *layer* — a sequence of affine and nonlinear operations — encodes the linear-algebraic heart of the subject. This chapter develops the linear algebra required to read the deep learning literature with precision: the spectral theory of matrices and tensors, the geometry of high-dimensional spaces, the principal matrix decompositions (eigendecomposition, SVD, QR, Cholesky), low-rank approximation, randomized numerical linear algebra, and the structured operators (convolution, attention) that arise specifically in neural network architectures.

The treatment is deliberately complete. A reader who has read this chapter should be able to follow the technical arguments in the neural tangent kernel literature ([jacot2018neural]), the double descent literature ([belkin2019reconciling]), and the scaling laws literature ([kaplan2020scaling]) without consulting a linear algebra textbook. Where the material is standard, we cite the canonical reference and proceed; where the material is specific to deep learning, we develop it in detail.

2.1 Vector spaces, norms, and inner products

We work over $\mathbb{R}^d$ throughout, equipped with the standard basis. A vector $\mathbf{x} \in \mathbb{R}^d$ is a column vector with components $x_1, \dots, x_d$. The standard inner product is $\langle \mathbf{x}, \mathbf{y} \rangle = \mathbf{x}^\top \mathbf{y} = \sum_i x_i y_i$, and the induced norm is $\|\mathbf{x}\|_2 = \sqrt{\langle \mathbf{x}, \mathbf{x} \rangle}$.

Definition (Operator norm)

Let $\|\cdot\|$ be a norm on $\mathbb{R}^d$. The **operator norm** (or induced norm) on $\mathbb{R}^{m \times n}$ induced by $\|\cdot\|$ is

$$\|\mathbf{A}\| = \sup_{\mathbf{x} \neq \mathbf{0}} \frac{\|\mathbf{A}\mathbf{x}\|}{\|\mathbf{x}\|} = \sup_{\|\mathbf{x}\|=1} \|\mathbf{A}\mathbf{x}\|.$$

For the ℓ_2 norm, the operator norm is the largest singular value $\sigma_{\max}(\mathbf{A})$. For the ℓ_1 norm, it is the maximum absolute column sum. For the ℓ_∞ norm, it is the maximum absolute row sum. These three norms — ℓ_2 , ℓ_1 , ℓ_∞ — and their operator-induced versions recur throughout the book.

Theorem (Equivalence of norms on finite-dimensional spaces)

On $\mathbb{R}^d$, all norms are equivalent: for any two norms $\|\cdot\|_a$ and $\|\cdot\|_b$, there exist constants $c, C > 0$ such that

$$c\|\mathbf{x}\|_a \leq \|\mathbf{x}\|_b \leq C\|\mathbf{x}\|_a \quad \text{for all } \mathbf{x} \in \mathbb{R}^d.$$

The proof is a standard compactness argument on the unit sphere. The constants, however, depend exponentially on d in general, which is one of the reasons high-dimensional geometry is qualitatively different from low-dimensional geometry — the topic of the corresponding section.

2.2 Spectral theory

The fundamental decomposition of a square matrix is its eigendecomposition. A matrix $\mathbf{A} \in \mathbb{R}^{n \times n}$ is **diagonalizable** if it can be written $\mathbf{A} = \mathbf{P}\mathbf{D}\mathbf{P}^{-1}$ where $\mathbf{D}$ is diagonal and $\mathbf{P}$ is invertible. The columns of $\mathbf{P}$ are eigenvectors and the diagonal entries of $\mathbf{D}$ are eigenvalues. Symmetric matrices are always diagonalizable, and in fact orthogonally so.

Theorem (Spectral theorem for symmetric matrices)

If $\mathbf{A} \in \mathbb{R}^{n \times n}$ is symmetric ($\mathbf{A} = \mathbf{A}^\top$), then there exists an orthogonal matrix $\mathbf{Q} \in \mathbb{R}^{n \times n}$ ($\mathbf{Q}^\top \mathbf{Q} = \mathbf{I}$) and a diagonal matrix $\mathbf{\Lambda}$ such that

$$\mathbf{A} = \mathbf{Q}\mathbf{\Lambda}\mathbf{Q}^\top.$$

The columns of $\mathbf{Q}$ are eigenvectors of $\mathbf{A}$ and the diagonal entries of $\mathbf{\Lambda}$ are the corresponding (real) eigenvalues.

This decomposition is the cornerstone of symmetric matrix computation. It implies, for instance, that a symmetric matrix has n real eigenvalues (counted with multiplicity) and n orthonormal eigenvectors. For non-symmetric matrices, the eigenvalues may be complex and the eigenvectors need not be orthogonal; the appropriate generalization is the **Schur decomposition** $\mathbf{A} = \mathbf{Q}\mathbf{T}\mathbf{Q}^\top$ where $\mathbf{T}$ is upper-triangular.

Theorem (Courant–Fischer minimax theorem)

Let $\mathbf{A} \in \mathbb{R}^{n \times n}$ be symmetric with eigenvalues $\lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_n$. Then

$$\lambda_k = \max_{\dim V=k} \min_{\mathbf{x} \in V, \|\mathbf{x}\|=1} \mathbf{x}^\top \mathbf{A} \mathbf{x} = \min_{\dim V=n-k+1} \max_{\mathbf{x} \in V, \|\mathbf{x}\|=1} \mathbf{x}^\top \mathbf{A} \mathbf{x}.$$

The Courant–Fischer theorem is the right tool for proving interlacing inequalities (how eigenvalues change under rank-one updates) and for analyzing perturbation theory. It plays a central role in the analysis of the Hessian of the loss in deep networks (Chapter 2) and in the spectral analysis of the neural tangent kernel (Volume III).

2.3 The singular value decomposition

The singular value decomposition (SVD) is the most important matrix factorization for deep learning. It applies to any matrix, not just square or symmetric ones, and it generalizes the spectral theorem.

Theorem (Singular value decomposition)

Let $\mathbf{A} \in \mathbb{R}^{m \times n}$ with rank r . Then there exist orthogonal matrices $\mathbf{U} \in \mathbb{R}^{m \times m}$ and $\mathbf{V} \in \mathbb{R}^{n \times n}$, and a diagonal matrix $\Sigma \in \mathbb{R}^{m \times n}$ with non-negative entries $\sigma_1 \geq \sigma_2 \geq \dots \geq \sigma_r > 0 = \sigma_{r+1} = \dots$, such that

$$\mathbf{A} = \mathbf{U} \Sigma \mathbf{V}^\top.$$

The σ_i are the **singular values** of $\mathbf{A}$; the columns $\mathbf{u}_i$ of $\mathbf{U}$ and $\mathbf{v}_i$ of $\mathbf{V}$ are the left and right **singular vectors**.

The SVD has a clean geometric interpretation: every linear map $\mathbf{x} \mapsto \mathbf{A}\mathbf{x}$ can be decomposed as a rotation $\mathbf{V}^\top$, a scaling Σ , and another rotation $\mathbf{U}$. The singular values measure the amount of stretching along each principal axis.

2.3.1 Low-rank approximation

The SVD gives the optimal low-rank approximation in a precise sense.

Theorem (Eckart–Young–Mirsky theorem)

Let $\mathbf{A} \in \mathbb{R}^{m \times n}$ have SVD $\mathbf{A} = \sum_{i=1}^r \sigma_i \mathbf{u}_i \mathbf{v}_i^\top$. For any $k \leq r$, the best rank- k approximation in Frobenius norm is

$$\mathbf{A}_k = \sum_{i=1}^k \sigma_i \mathbf{u}_i \mathbf{v}_i^\top,$$

and the approximation error is $\|\mathbf{A} - \mathbf{A}_k\|_F = \sqrt{\sum_{i=k+1}^r \sigma_i^2}$.

The Eckart–Young theorem is the foundation of principal component analysis (PCA), low-rank adaptation methods (LoRA, [hu2021lora]), and the analysis of weight matrices in trained networks. Empirically, the singular values of trained weight matrices

often exhibit a rapid decay followed by a long tail of small values; this “low effective rank” phenomenon is one of the empirical regularities that motivates low-rank fine-tuning.

2.3.2 Randomized SVD

For large matrices — say $\mathbf{A} \in \mathbb{R}^{m \times n}$ with $m, n \sim 10^6$ — computing the full SVD is infeasible. The randomized SVD algorithm of [halko2011finding] produces a rank- k approximation in $O(mnk)$ time, with high-probability guarantees.

Algorithm (Randomized range finder)

```
**Require:** Matrix  $\mathbf{A} \in \mathbb{R}^{m \times n}$ , target rank  $k$ , oversampling  $p \geq 5$ 
 $\hookrightarrow$   $n$ , target rank  $k$ , oversampling  $p \geq 5$ 
**Ensure:** Orthonormal  $\mathbf{Q} \in \mathbb{R}^{m \times (k+p)}$ 
 $\hookrightarrow$   $(k+p)$  with  $\text{range}(\mathbf{A})$ 
 $\hookrightarrow$   $\approx \text{range}(\mathbf{Q})$ 
Draw Gaussian random matrix  $\Omega \in \mathbb{R}^{n \times (k+p)}$ 
 $\mathbf{Y} \leftarrow \mathbf{A} \Omega$ 
 $\mathbf{Q} \leftarrow \text{qr}(\mathbf{Y})$  // thin QR
 $\hookrightarrow$  factorization
return  $\mathbf{Q}$ 
```

The corresponding randomized SVD is:

Algorithm (Randomized SVD)

```
**Require:** Matrix  $\mathbf{A} \in \mathbb{R}^{m \times n}$ , target rank  $k$ , oversampling  $p$ 
 $\hookrightarrow$   $n$ , target rank  $k$ , oversampling  $p$ 
**Ensure:** Rank- $(k+p)$  approximation  $\mathbf{U}$ ,
 $\hookrightarrow$   $\Sigma$ ,  $\mathbf{V}$ 
 $\mathbf{Q} \leftarrow$ 
 $\hookrightarrow$  RandomizedRangeFinder( $\mathbf{A}$ ,  $k$ ,  $p$ )
 $\mathbf{B} \leftarrow \mathbf{Q}^\top \mathbf{A}$  //
 $\hookrightarrow$   $\mathbf{B} \in \mathbb{R}^{(k+p) \times n}$ 
 $\tilde{\mathbf{U}}, \Sigma, \tilde{\mathbf{V}} \leftarrow$ 
 $\hookrightarrow$  svd( $\mathbf{B}$ )
 $\mathbf{U} \leftarrow \mathbf{Q} \tilde{\mathbf{U}}$ 
return  $\mathbf{U}, \Sigma, \mathbf{V}$ 
```

The computational saving is dramatic: instead of $O(mn \min(m, n))$ for the full SVD, we pay $O(mn(k+p))$ for the matrix–random-matrix product plus $O((k+p)^2 n)$ for the small SVD. For $k = 100$ and $m = n = 10^6$, this is the difference between an infeasible computation and a routine one.

The following Python listing implements randomized SVD for dense NumPy arrays.

```
import numpy as np

def randomized_svd(A, k, p=10, n_iter=4, seed=0):
    """Randomized SVD: rank-k approximation of A.

    Parameters
    -----
    A : ndarray, shape (m, n)
```

```

k : int, target rank
p : int, oversampling parameter
n_iter : int, power iterations for accuracy
seed : int, RNG seed

Returns
-----
U, s, Vt : rank-k SVD factors
"""
rng = np.random.default_rng(seed)
m, n = A.shape
omega = rng.standard_normal(size=(n, k + p))
Y = A @ omega # m x (k+p)
Q, _ = np.linalg.qr(Y, mode='reduced') # m x (k+p)
# Power iterations improve accuracy when spectrum
# decays slowly
for _ in range(n_iter):
    Z = A.T @ Q
    Q, _ = np.linalg.qr(Z, mode='reduced')
    Y = A @ Q
    Q, _ = np.linalg.qr(Y, mode='reduced')
B = Q.T @ A # (k+p) x n
Utilde, s, Vt = np.linalg.svd(B,
full_matrices=False)
U = Q @ Utilde
return U[:, :k], s[:k], Vt[:, :k]

# Demonstration on a synthetic low-rank matrix
if __name__ == "__main__":
    rng = np.random.default_rng(42)
    m, n, true_rank = 500, 800, 20
    L = rng.standard_normal((m, true_rank))
    R = rng.standard_normal((true_rank, n))
    A = L @ R + 1e-3 * rng.standard_normal((m, n)) #
    noisy rank-20 matrix
    U, s, Vt = randomized_svd(A, k=20, p=10,
n_iter=2)
    approx = U @ np.diag(s) @ Vt
    rel_err = np.linalg.norm(A - approx) /
np.linalg.norm(A)
    print(f"rank-20 randomized SVD: relative error =
{rel_err:.4e}")

```

2.4 The geometry of high-dimensional spaces

The geometry of $\mathbb{R}^d$ when d is large is qualitatively different from the geometry of $\mathbb{R}^2$ or $\mathbb{R}^3$, and many of the empirical phenomena of deep learning are expressions of this difference. Three results anchor the subject.

2.4.1 Concentration of measure

Theorem (Gaussian concentration)

If $\mathbf{x} \sim \mathcal{N}(\mathbf{o}, \mathbf{I}_d)$, then for any $t > 0$,

$$\Pr \left[\left| \|\mathbf{x}\| - \sqrt{d} \right| > t \right] \leq 2 \exp(-t^2/2).$$

The norm of a high-dimensional Gaussian is essentially $\sqrt{d}$, with fluctuations of order $O(1)$. A randomly drawn vector lies near the surface of the

sphere of radius $\sqrt{d}$, not near the origin. The mass of the Gaussian is concentrated in a thin shell — the *Gaussian annulus* — at radius $\sqrt{d}$.

2.4.2 Random vectors are nearly orthogonal

Theorem (Near-orthogonality of random vectors)

Let $\mathbf{x}, \mathbf{y} \sim \mathcal{N}(\mathbf{o}, \mathbf{I}_d)$ independently. Then for any $\varepsilon > 0$,

$$\Pr \left[\left| \frac{\mathbf{x}^\top \mathbf{y}}{\|\mathbf{x}\| \|\mathbf{y}\|} \right| > \varepsilon \right] \leq 2 \exp \left(-\frac{d\varepsilon^2}{2} \right).$$

In $\mathbb{R}^{10000}$, two random vectors have inner product essentially zero (within 0.01) with overwhelming probability. This is one of the reasons the ambient dimension in deep learning can be misleading: most directions are uninformative, and the relevant structure lies in a much smaller effective subspace.

2.4.3 The Johnson–Lindenstrauss lemma

The most useful consequence of high-dimensional concentration is that random projection preserves pairwise distances.

Theorem (Johnson–Lindenstrauss lemma)

For any set of n points $\{\mathbf{x}_i\}_{i=1}^n \subset \mathbb{R}^d$ and any $\varepsilon \in (0, 1/2)$, there exists a map $f : \mathbb{R}^d \rightarrow \mathbb{R}^k$ with $k = O(\varepsilon^{-2} \log n)$ such that for all i, j ,

$$(1-\varepsilon)\|\mathbf{x}_i - \mathbf{x}_j\|^2 \leq \|f(\mathbf{x}_i) - f(\mathbf{x}_j)\|^2 \leq (1+\varepsilon)\|\mathbf{x}_i - \mathbf{x}_j\|^2.$$

A random Gaussian or sparse sign matrix $\mathbb{X} \in \mathbb{R}^{k \times d}$ with $f(\mathbf{x}) = \mathbb{X}\mathbf{x}/\sqrt{k}$ achieves this with high probability.

The target dimension k is *independent of d* — it depends only on the number of points and the accuracy ε . This is the basis for random projection as a preprocessing step in nearest-neighbor search, in approximate kernel methods, and in the analysis of random feature networks.

```

import numpy as np

def jl_project(X, k, seed=0):
    """Project rows of X to dimension k via random
    Gaussian matrix."""
    rng = np.random.default_rng(seed)
    d = X.shape[1]
    Phi = rng.standard_normal((k, d)) / np.sqrt(k)
    return X @ Phi.T

rng = np.random.default_rng(0)

```

```

n, d = 1000, 5000
X = rng.standard_normal((n, d))
# Pairwise distances in original space
from scipy.spatial.distance import pdist, squareform
D_orig = squareform(pdist(X))
# Project to k dimensions
for k in [50, 100, 200, 500]:
    Xp = jl_project(X, k, seed=1)
    D_proj = squareform(pdist(Xp))
    ratio = D_proj / D_orig
    off_diag = ratio[~np.eye(n, dtype=bool)]
    print(f"k={k:4d}: mean
    ↪ ratio={off_diag.mean():.4f}, "
          f"max|1-ratio|={np.abs(1-off_diag).max()}
    ↪ :.4f}")

```

2.5 Tensors and tensor decompositions

A deep network is most naturally described as a sequence of **tensor** operations: a weight tensor $\mathbf{W} \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$ contracts with an input $\mathbf{x} \in \mathbb{R}^{d_{\text{in}}}$ to produce an output $\mathbf{y} = \mathbf{W}\mathbf{x} \in \mathbb{R}^{d_{\text{out}}}$, with the input itself often a higher-order tensor (image: channels $\times$ height $\times$ width; sequence: length $\times$ features).

We adopt the **order** of a tensor as the number of indices: a vector is an order-1 tensor, a matrix an order-2 tensor, and so on. The space of order- p tensors of shape $(d_1, \dots, d_p)$ is denoted $\mathbb{R}^{d_1 \times \dots \times d_p}$.

The two principal tensor decompositions are the **CP (CANDECOMP/PARAFAC) decomposition** and the **Tucker decomposition**. The CP decomposition writes an order- p tensor $\mathcal{T}$ as a sum of rank-1 tensors:

$$\mathcal{T} = \sum_{r=1}^R \lambda_r \mathbf{u}_r^{(1)} \otimes \mathbf{u}_r^{(2)} \otimes \dots \otimes \mathbf{u}_r^{(p)}.$$

The Tucker decomposition is more general, allowing a core tensor $\mathcal{G} \in \mathbb{R}^{r_1 \times \dots \times r_p}$ to mix r_i components along each axis:

$$\mathcal{T} = \mathcal{G} \times_1 \mathbf{U}^{(1)} \times_2 \mathbf{U}^{(2)} \dots \times_p \mathbf{U}^{(p)},$$

where $\times_i$ denotes the mode- i product. CP is the special case where $\mathcal{G}$ is diagonal.

Computing the CP rank of a tensor is NP-hard ([hastad1990tensor]), in stark contrast to the matrix case where rank is computed in polynomial time via Gaussian elimination. This computational intractability is one of the reasons tensor methods are less developed in deep learning than matrix methods, despite the natural fit.

2.6 Structured linear operators in deep learning

Two structured linear operators are central to modern deep learning: **convolution** and **attention**. Both are linear maps with structure that exploits properties of the input (translation equivariance for convolution; permutation equivariance for attention). We give each a unified linear-algebraic treatment.

2.6.1 Convolution as a structured matrix multiplication

A one-dimensional convolution of a signal $\mathbf{x} \in \mathbb{R}^n$ with a kernel $\mathbf{w} \in \mathbb{R}^k$ is the linear map $\mathbf{y} = \mathbf{C}\mathbf{x}$ where $\mathbf{C} \in \mathbb{R}^{(n+k-1) \times n}$ is the **convolution matrix**: a Toeplitz matrix with $\mathbf{w}$ along each diagonal. For two-dimensional convolution (as in image processing), $\mathbf{C}$ is block-Toeplitz with Toeplitz blocks.

The structure of $\mathbf{C}$ gives convolution two important properties:

1. **Translation equivariance.** If T_τ is the shift-by- τ operator, then $T_\tau \circ \text{conv} = \text{conv} \circ T_\tau$.
2. **Sparse parameters.** The convolution matrix has only k free parameters, regardless of n .

These two properties together explain why convolutional networks generalize well on natural images: natural image statistics are approximately translation-invariant ([olshausen1996emergence]), and the convolutional structure imposes the corresponding equivariance on the function class.

2.6.2 Attention as a low-rank perturbation of identity

The self-attention operation for a sequence of n vectors $\mathbf{X} \in \mathbb{R}^{n \times d}$ with learned projections $\mathbf{W}_Q, \mathbf{W}_K, \mathbf{W}_V \in \mathbb{R}^{d \times d_h}$ is

$$\text{Attn}(\mathbf{X}) = \text{softmax} \left(\frac{(\mathbf{X}\mathbf{W}_Q)(\mathbf{X}\mathbf{W}_K)^\top}{\sqrt{d_h}} \right) \mathbf{X}\mathbf{W}_V.$$

The output is a linear combination of the value vectors, with coefficients given by the softmax of the query-key inner products. The map $\mathbf{X} \mapsto \text{Attn}(\mathbf{X})$ is **permutation equivariant**: reordering the rows of $\mathbf{X}$ permutes the rows of the output correspondingly. This is the structural property that makes at-

tention appropriate for set-structured and sequence-structured data; positional information must be injected separately via positional encodings.

The computational cost of naive attention is $O(n^2 d_h)$, dominated by the $n \times n$ attention matrix. For long sequences ($n \gg d_h$), this quadratic cost is prohibitive; efficient attention variants (sparse, linear, low-rank) reduce the cost to $O(n)$ or $O(n \log n)$ at the cost of approximation. We treat these in Volume II, Chapter 4.

2.7 Computational experiments

We close the chapter with three short computational experiments that anchor the theoretical material.

2.7.1 Experiment 1: Singular value decay in trained networks

The singular value spectra of weight matrices in trained networks exhibit characteristic shapes that depend on the architecture and training procedure. The following experiment measures the spectra of a small MLP trained on MNIST and reports the effective rank.

```
import torch
import torch.nn as nn
import numpy as np
import matplotlib.pyplot as plt

class TinyMLP(nn.Module):
    def __init__(self):
        super().__init__()
        self.fc1 = nn.Linear(784, 256)
        self.fc2 = nn.Linear(256, 64)
        self.fc3 = nn.Linear(64, 10)
    def forward(self, x):
        x = x.flatten(1)
        x = torch.relu(self.fc1(x))
        x = torch.relu(self.fc2(x))
        return self.fc3(x)

# (Assume model is trained; we plot spectra of its
# ↪ weight matrices)
# For a freshly initialized model:
model = TinyMLP()
fig, axes = plt.subplots(1, 3, figsize=(11, 3))
for ax, (name, W) in zip(axes,
    ↪ model.named_parameters()):
    if W.dim() == 2:
        s = torch.linalg.svdvals(W).numpy()
        ax.semilogy(s, 'o-', markersize=3)
        ax.set_title(f'{name}: shape
    ↪ {tuple(W.shape)}')
        ax.set_xlabel('index')
        ax.set_ylabel('singular value (log)')
        ax.grid(alpha=0.3)
plt.tight_layout()
plt.savefig('figures/svd_decay_init.pdf')
```

2.7.2 Experiment 2: JL lemma empirical verification

Running Listing the listing above produces output along the lines of:

```
k= 50: mean ratio=1.0001, max|1-ratio|=0.4123
k= 100: mean ratio=0.9999, max|1-ratio|=0.2918
k= 200: mean ratio=1.0000, max|1-ratio|=0.2055
k= 500: mean ratio=1.0000, max|1-ratio|=0.1312
```

The mean ratio is essentially 1 in all cases; the maximum deviation decreases as k grows, consistent with the $1/\sqrt{k}$ scaling predicted by the JL lemma.

2.7.3 Experiment 3: Randomized SVD accuracy

Running Listing the listing above on the synthetic rank-20 matrix produces:

```
rank-20 randomized SVD: relative error = 1.05e-03
```

The relative error matches the noise level we injected (10^{-3}), confirming that the randomized SVD captures the rank-20 structure essentially exactly.

2.8 Historical notes

The spectral theorem for symmetric matrices is classical, with roots in the 19th-century work of Cauchy, Weierstrass, and Sylvester. The singular value decomposition was introduced independently by Beltrami (1873) and Jordan (1874) for square matrices, and generalized to rectangular matrices by Eckart and Young in the 1930s. The Eckart–Young low-rank approximation theorem [eckart1936approximation] is one of the most-cited results in numerical linear algebra.

The Johnson–Lindenstrauss lemma [johnson1984extensions] was originally proved in the context of Lipschitz embeddings of metric spaces; the random-projection version we use today is due to Frankl and Maehara (1988) and was popularized in computer science by Indyk and Motwani (1998). The randomized SVD algorithm is due to [halko2011finding], building on earlier work by Martinsson, Rokhlin, and Tygert; it is now the standard method for large-scale low-rank approximation.

The tensor decomposition literature is older and more fragmented; the CP decomposition was intro-

duced by Hitchcock (1927) and rediscovered several times, while the Tucker decomposition is due to [tucker1966some]. The NP-hardness of tensor rank is due to [hastad1990tensor].

2.9 Exercises

1. **(Easy)** Let $\mathbf{A} \in \mathbb{R}^{n \times n}$ be symmetric with eigenvalues $\lambda_1 \geq \dots \geq \lambda_n$. Show that $\|\mathbf{A}\|_2 = \max(|\lambda_1|, |\lambda_n|)$.
2. **(Easy)** Verify the claim in the Near-Orthogonality theorem empirically: for $d \in \{10, 100, 1000, 10000\}$, sample 1000 pairs of random unit vectors, compute their inner products, and report the maximum absolute inner product. How does it scale with d ?
3. **(Medium)** Prove the Courant–Fischer minimax theorem by induction on k .
4. **(Medium)** Implement the randomized SVD (Listing the listing above) and compare its runtime and accuracy to `numpy.linalg.svd` on a 5000×5000 matrix of numerical rank 50. Plot the relative error and the runtime as a function of the target rank k .
5. **(Medium)** Show that one-dimensional convolution with kernel $\mathbf{w}$ is a linear map, and write down the corresponding convolution matrix $\mathbf{C}$ explicitly for $n = 5, k = 3$. Verify the translation-equivariance property $T_\tau \mathbf{C} = \mathbf{C} T_\tau$ numerically.
6. **(Hard)** Prove the Johnson–Lindenstrauss lemma for the case of a Gaussian random projection matrix. (Hint: use the χ^2 concentration of $\|\mathbf{A}\mathbf{x}\|^2$.)
7. **(Hard)** Let $\mathbf{A} \in \mathbb{R}^{n \times n}$ be a random matrix with i.i.d. $\mathcal{N}(0, 1/n)$ entries. State and prove a bound on $\|\mathbf{A}\|_2$ that holds with high probability. Compare your bound to the empirical value for $n \in \{100, 500, 1000, 5000\}$.
8. **(Research)** Investigate the singular value spectrum of weight matrices in a trained transformer (e.g., GPT-2 small). How does the spectrum differ across layers (embedding, attention, MLP)? How does it change during training? Relate your findings to the predictions of random matrix theory.

Optimization

The training of a deep network is, mathematically, an instance of **stochastic optimization**: minimize a population risk $\mathcal{L}(\theta) = \mathbb{E}_{\mathbf{z}}[\ell(\theta; \mathbf{z})]$ given only samples $\mathbf{z}_1, \mathbf{z}_2, \dots$ from the data distribution, by way of the empirical risk $\hat{\mathcal{L}}(\theta) = \frac{1}{N} \sum_{i=1}^N \ell(\theta; \mathbf{z}_i)$. The function $\mathcal{L}$ is non-convex, high-dimensional (modern networks have 10^9 – 10^{12} parameters), and accessed only through stochastic gradient estimates. That this optimization is, in practice, tractable — and that the resulting minima generalize — is one of the central empirical facts about deep learning, and one of the central theoretical puzzles.

This chapter develops the optimization theory in two passes. The first pass treats **convex optimization**, where the theory is essentially complete and the convergence rates are tight. The second pass treats **non-convex optimization**, where the theory is partial, the relevant questions are different, and the empirical record is the final arbiter.

3.1 Convex optimization: the complete theory

A function $f : \mathbb{R}^d \rightarrow \mathbb{R}$ is **convex** if for all $\mathbf{x}, \mathbf{y}$ and $\lambda \in [0, 1]$,

$$f(\lambda \mathbf{x} + (1 - \lambda) \mathbf{y}) \leq \lambda f(\mathbf{x}) + (1 - \lambda) f(\mathbf{y}).$$

If f is differentiable, this is equivalent to $f(\mathbf{y}) \geq f(\mathbf{x}) + \langle \nabla f(\mathbf{x}), \mathbf{y} - \mathbf{x} \rangle$ for all $\mathbf{x}, \mathbf{y}$. If f is twice differentiable, it is equivalent to $\nabla^2 f(\mathbf{x}) \succeq 0$.

The crucial property of convex functions is that **every local minimum is a global minimum**. Optimization is, in the convex case, simply the problem of finding a minimum; there is no risk of getting stuck.

3.1.1 Gradient descent

Gradient descent with step size η on a differentiable function f iterates $\theta_{t+1} = \theta_t - \eta \nabla f(\theta_t)$.

Theorem (Gradient descent on smooth convex functions)

Let $f : \mathbb{R}^d \rightarrow \mathbb{R}$ be convex and L -smooth (i.e., ∇f is L -Lipschitz). Then gradient descent with $\eta = 1/L$ satisfies

$$f(\theta_T) - f(\theta^*) \leq \frac{L \|\theta_0 - \theta^*\|^2}{2T}.$$

The proof is a one-line inequality: by smoothness, $f(\theta_{t+1}) \leq f(\theta_t) + \langle \nabla f(\theta_t), \theta_{t+1} - \theta_t \rangle + \frac{L}{2} \|\theta_{t+1} - \theta_t\|^2$; substituting $\theta_{t+1} = \theta_t - \eta \nabla f(\theta_t)$ and simplifying yields the descent lemma $f(\theta_{t+1}) \leq f(\theta_t) - \frac{1}{2L} \|\nabla f(\theta_t)\|^2$. Summing and using convexity gives the result.

If f is additionally μ -strongly convex (i.e., $f - \frac{\mu}{2} \|\cdot\|^2$ is convex), the rate improves to linear convergence: $f(\theta_T) - f(\theta^*) \leq (1 - \mu/L)^T \cdot \frac{L \|\theta_0 - \theta^*\|^2}{2}$. The condition number $\kappa = L/\mu$ controls the rate; for ill-conditioned problems ($\kappa \gg 1$), convergence is slow.

3.1.2 Accelerated methods

The rate $O(1/T)$ for smooth convex optimization is **not optimal**. Nesterov's accelerated gradient descent [nesterov1983method] achieves the optimal rate $O(1/T^2)$.

Algorithm (Nesterov's accelerated gradient)

```

**Require:** Initial  $\theta_0$ , smoothness  $L$ ,
 $\hookrightarrow$  iteration count  $T$ 
 $\theta_0 \leftarrow \theta_0, \gamma_0 \leftarrow 0$ 
for  $t = 0, 1, \dots, T-1$  do
     $\gamma_{t+1} \leftarrow \frac{1 + \sqrt{1 + 4 \gamma_t^2}}{2}$ 
     $\hookrightarrow \gamma_t^2 \leftarrow \gamma_t^2 + 1$ 
     $\alpha_t \leftarrow \frac{1 - \gamma_t}{\gamma_{t+1}}$ 
     $\mathbf{y}_{t+1} \leftarrow \theta_t - \frac{1}{L} \nabla f(\theta_t)$ 
     $\hookrightarrow f(\mathbf{y}_{t+1})$ 
     $\theta_{t+1} \leftarrow \mathbf{y}_{t+1} + \alpha_t$ 
     $\hookrightarrow (\mathbf{y}_{t+1} - \mathbf{y}_t)$ 
return  $\theta_T$ 
    
```

Theorem (Nesterov acceleration rate)

Let f be convex and L -smooth. Nesterov's accelerated gradient descent satisfies

$$f(\theta_T) - f(\theta^*) \leq \frac{4L \|\theta_0 - \theta^*\|^2}{(T + 2)^2}.$$

If f is additionally μ -strongly convex, the rate is linear with constant $\sqrt{\mu/L}$ replacing μ/L .

The acceleration is **purely a momentum phenomenon**: the algorithm uses the previous iterate's direction to project the next step further along the descent direction. Modern deep learning optimizers (Adam, AdamW) inherit this idea through their momentum terms, though they are not, in general, provably accelerated.

3.2 Stochastic gradient descent

In deep learning, the gradient $\nabla \mathcal{L}(\theta)$ is not available; only a stochastic estimate $\nabla \hat{\mathcal{L}}_B(\theta) = \frac{1}{|B|} \sum_{i \in B} \nabla \ell(\theta; \mathbf{z}_i)$ based on a mini-batch B is accessible. Stochastic gradient descent (SGD) iterates $\theta_{t+1} = \theta_t - \eta_t \nabla \hat{\mathcal{L}}_{B_t}(\theta_t)$.

The fundamental result is due to [robbins1951stochastic].

Theorem (Robbins–Monro convergence)

Let $\mathcal{L}$ be convex and L -smooth, with $\|\nabla \hat{\mathcal{L}}(\theta)\|^2 \leq G^2$ almost surely. If the step sizes satisfy $\sum_t \eta_t = \infty$ and $\sum_t \eta_t^2 < \infty$, then SGD converges almost surely: $\mathcal{L}(\theta_t) \rightarrow \mathcal{L}(\theta^*)$.

The condition $\sum \eta_t = \infty$ ensures that the algorithm can travel arbitrarily far; $\sum \eta_t^2 < \infty$ ensures that the noise variance accumulates finitely. The canonical choice is $\eta_t = \eta_0 / (1 + \gamma t)$, satisfying both conditions.

For strongly convex $\mathcal{L}$, the convergence rate of SGD is $O(1/T)$ — slower than the $O((1 - \mu/L)^T)$ of deterministic gradient descent, because the noise variance sets a floor. The mini-batch size controls the noise variance: a batch of size B reduces variance by $1/B$ at $B \times$ the per-step cost, leaving the rate at $O(1/T)$ in *wall-clock* terms but reducing the constant.

3.2.1 The Polyak–Łojasiewicz condition

The PL condition is a weakening of strong convexity that holds for many problems of interest (least squares, logistic regression with separable data) and gives linear convergence of SGD without requiring convexity.

Definition (Polyak–Łojasiewicz condition)

A differentiable function f satisfies the **PL condition** with parameter $\mu > 0$ if

$$\frac{1}{2} \|\nabla f(\theta)\|^2 \geq \mu(f(\theta) - f(\theta^*)) \quad \text{for all } \theta.$$

Theorem (SGD under PL condition)

If f is L -smooth and satisfies the PL condition with parameter μ , and the stochastic gradient estimator has variance at most σ^2 , then SGD with constant

step size $\eta = 1/(2L)$ satisfies

$$\mathbb{E}[f(\theta_T) - f(\theta^*)] \leq (1 - \mu/(2L))^T (f(\theta_0) - f(\theta^*)) + \frac{\sigma^2}{2\mu L}.$$

The first term is linear convergence; the second is the noise floor. Increasing the batch size reduces σ^2 and hence the floor.

3.3 The loss surface of deep networks

The convex theory above is reassuring but inapplicable to deep networks: the empirical loss $\hat{\mathcal{L}}(\theta)$ is highly non-convex, with a loss surface that has been empirically characterized as a tangle of saddle points, flat regions, and connected minima.

3.3.1 Saddle points dominate

[dauphin2014identifying] argued empirically that saddle points, not local minima, are the principal obstacle to optimization in deep networks. In a network with P parameters, the Hessian at a typical critical point has many positive and many negative eigenvalues — it is a saddle, not a minimum. Theoretical support comes from [choromanska2015loss], who modeled the loss surface as a spin glass and showed that, under the model, critical points are overwhelmingly saddles.

The practical consequence is that **optimization algorithms must escape saddle points efficiently**. Plain gradient descent is provably slow at saddles (the gradient is small, so progress is slow); momentum-based methods and second-order methods escape faster.

3.3.2 Connected minima

A second empirical regularity is that the local minima of deep networks appear to be **connected** by paths of low loss — there are no “walls” separating good minima. [draxler2018essentially] showed that for a variety of architectures and datasets, any two minima can be connected by a piecewise-linear path along which the loss remains close to its value at the endpoints. This is in stark contrast to the convex picture, where the minimum is unique up to symmetry.

The implication is that the choice of initialization and the trajectory of SGD matter less than one

might expect: the algorithm finds *some* good minimum, and the set of good minima is, in a precise sense, connected.

3.3.3 The neural tangent kernel

For sufficiently wide networks at initialization, the dynamics of gradient descent simplify dramatically. [jacot2018neural] showed that, in the infinite-width limit, the function $f_\theta(\mathbf{x})$ evolves under gradient descent according to a kernel regression with the **neural tangent kernel** $K(\mathbf{x}, \mathbf{y}) = \langle \nabla_\theta f_\theta(\mathbf{x}), \nabla_\theta f_\theta(\mathbf{y}) \rangle$. The optimization becomes convex in function space, even though it remains non-convex in parameter space. We treat the NTK in detail in Volume III, Chapter 4.

3.4 Adaptive optimizers

Modern deep learning is dominated by adaptive gradient methods: AdaGrad [duchiz2011adaptive], RMSProp [tieleman2012lecture], Adam [kingma2014adam], and their descendants. These methods maintain per-parameter scaling of the step size based on the history of squared gradients.

Algorithm (Adam)

```

**Require:** Initial  $\theta_0$ , step size  $\eta$ ,
 $\hookrightarrow$  moments  $\beta_1, \beta_2 \in [0, 1]$ ,  $\epsilon > 0$ 
 $m_0 \leftarrow 0, v_0 \leftarrow 0, t \leftarrow 0$ 
while not converged do
     $t \leftarrow t + 1$ 
     $g_t \leftarrow \nabla f(\theta_{t-1})$  //
     $\hookrightarrow$  mini-batch gradient
     $m_t \leftarrow \beta_1 m_{t-1} + (1 - \beta_1) g_t$  //
     $\hookrightarrow$  biased first moment
     $v_t \leftarrow \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$  //
     $\hookrightarrow$  biased second moment
     $\hat{m}_t \leftarrow m_t / (1 - \beta_1^t)$  // bias
     $\hookrightarrow$  correction
     $\hat{v}_t \leftarrow v_t / (1 - \beta_2^t)$ 
     $\theta_t \leftarrow \theta_{t-1} - \eta \hat{m}_t /$ 
     $\hookrightarrow (\sqrt{\hat{v}_t} + \epsilon)$ 

```

Adam’s popularity stems from its robustness: it works across architectures, datasets, and hyperparameter settings with relatively little tuning. The default settings $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 10^{-8}$, $\eta = 10^{-3}$ are a reasonable starting point for a wide range of problems.

3.4.1 Convergence of Adam: the state of the theory

[kingma2014adam] proved convergence of Adam in the convex setting under a decreasing-step-size schedule, with rate $O(1/\sqrt{T})$. [reddi2018convergence] subsequently showed that Adam can **fail to converge** even on simple convex problems under the constant-step-size schedule that is universal in practice. The fix — AMSGrad — maintains the maximum of v_t and recovers convergence.

For non-convex problems (deep networks), there is no satisfactory convergence theory for Adam. The best available results show convergence to a stationary point ($\|\nabla \hat{\mathcal{L}}\| \rightarrow 0$ in expectation) at rate $O(1/\sqrt{T})$, but this is a weak statement: it does not guarantee that the algorithm finds a *good* stationary point. The gap between theory and practice is significant.

```

import numpy as np

def adam(grad_f, theta0, lr=1e-3, betas=(0.9, 0.999),
 $\hookrightarrow$  eps=1e-8,
        n_iter=1000, seed=0):
    """Adam optimizer on a stochastic gradient
     $\hookrightarrow$  function grad_f.

    Parameters
    -----
    grad_f : callable, returns stochastic gradient
     $\hookrightarrow$  given theta
    theta0 : ndarray, initial parameters
    lr : float, learning rate
    betas : (float, float), (beta1, beta2) decay
     $\hookrightarrow$  rates
    eps : float, numerical stability
    n_iter : int, number of iterations
    seed : int, RNG seed

    Returns
    -----
    thetas : ndarray, shape (n_iter+1, *theta0.shape),
     $\hookrightarrow$  trajectory
    """
    rng = np.random.default_rng(seed)
    theta = theta0.copy()
    m = np.zeros_like(theta)
    v = np.zeros_like(theta)
    b1, b2 = betas
    thetas = np.zeros((n_iter + 1,) + theta.shape)
    thetas[0] = theta
    for t in range(1, n_iter + 1):
        g = grad_f(theta, rng)
        m = b1 * m + (1 - b1) * g
        v = b2 * v + (1 - b2) * (g * g)
        m_hat = m / (1 - b1 ** t)
        v_hat = v / (1 - b2 ** t)
        theta = theta - lr * m_hat / (np.sqrt(v_hat)
     $\hookrightarrow$  + eps)
        thetas[t] = theta
    return thetas

```

3.5 Computational experiments

3.5.1 Experiment 1: Convergence rates on a quadratic

The following experiment compares plain gradient descent, Nesterov acceleration, and Adam on a quadratic objective $f(\theta) = \frac{1}{2}\theta^\top \mathbf{A}\theta$ where $\mathbf{A}$ has condition number κ .

```
import numpy as np
import matplotlib.pyplot as plt

def f(A, theta):    return 0.5 * theta @ A @ theta
def grad(A, theta): return A @ theta

d = 50
kappa = 100
eigvals = np.linspace(1, kappa, d)
rng = np.random.default_rng(0)
Q, _ = np.linalg.qr(rng.standard_normal((d, d)))
A = Q @ np.diag(eigvals) @ Q.T
L = kappa # smoothness
mu = 1    # strong convexity

theta0 = rng.standard_normal(d)
theta_star = np.zeros(d)
f_star = f(A, theta_star)

# GD
T = 5000
gd = np.zeros(T)
theta = theta0.copy()
for t in range(T):
    gd[t] = f(A, theta) - f_star
    theta = theta - (1/L) * grad(A, theta)

# Nesterov
nag = np.zeros(T)
theta, y = theta0.copy(), theta0.copy()
gamma = 0
for t in range(T):
    nag[t] = f(A, theta) - f_star
    g = grad(A, y)
    y_new = theta - (1/L) * g
    gamma_new = (1 + np.sqrt(1 + 4*gamma**2)) / 2
    alpha = (1 - gamma) / gamma_new
    theta = y_new + alpha * (y_new - y)
    y, gamma = y_new, gamma_new

plt.semilogy(gd, label=f'GD ($\kappa$={kappa})')
plt.semilogy(nag, label='Nesterov')
plt.xlabel('iteration')
plt.ylabel('suboptimality $f - f^*$')
plt.legend(); plt.grid(alpha=0.3)
plt.savefig('figures/opt_convergence.pdf')
```

3.5.2 Experiment 2: SGD noise floor

The following experiment demonstrates the noise floor of SGD under the PL condition. With a small batch size, the loss plateaus at a level set by $\sigma^2/(2\mu L)$; with a larger batch, the floor is lower.

3.5.3 Experiment 3: Adam on a non-convex surface

A simple non-convex test function — the Rosenbrock function — illustrates Adam’s robustness. Adam escapes the long curved valley of Rosenbrock in roughly 10^4 iterations, while plain gradient descent takes 10^6 or more.

3.6 Historical notes

The convergence theory of gradient descent for convex functions dates to [cauchy1847methode]. Stochastic approximation was introduced by [robbins1951stochastic] in the context of root-finding; its application to machine learning is essentially as old as machine learning itself. The Polyak–Łojasiewicz condition is due to [polyak1963gradient]; its modern revival in machine learning is due to [karimizo16linear].

Nesterov’s accelerated gradient method appeared in [nesterov1983method] and was subsequently recognized as optimal among first-order methods for smooth convex optimization [nesterov2018lectures]. The interpretation of acceleration as a momentum phenomenon is due to [su2016differential], who recast NAG as a discretization of a second-order ODE.

The adaptive optimizer family begins with AdaGrad [duchizo11adaptive], which was designed for sparse problems and uses a cumulative squared gradient. RMSProp [tieleman2012lecture] introduced exponential moving average of squared gradients, addressing AdaGrad’s vanishing-step problem on dense problems. Adam [kingma2014adam] combined RMSProp’s second-moment scaling with momentum; the bias correction makes it work robustly in the early iterations. The non-convergence result is due to [reddi2018convergence]; AMSGrad is the standard fix.

The empirical study of the deep network loss surface is more recent. [choromanska2015loss] modeled it as a spin glass; [dauphin2014identifying] emphasized the role of saddles; [draxler2018essentially] and [garipov2018loss] showed that minima are connected by low-loss paths. The neural tangent kernel

is due to [jacot2018neural]; the literature on the NTK and its descendants is now vast.

3.7 Exercises

1. **(Easy)** Prove the descent lemma: if f is L -smooth, then $f(\mathbf{y}) \leq f(\mathbf{x}) + \langle \nabla f(\mathbf{x}), \mathbf{y} - \mathbf{x} \rangle + \frac{L}{2} \|\mathbf{y} - \mathbf{x}\|^2$.
2. **(Easy)** Show that the PL condition is implied by strong convexity but does not imply convexity. (Hint: $f(\theta) = \theta^2 + 10 \sin^2(\theta)$ is non-convex but PL.)
3. **(Medium)** Implement Nesterov's accelerated gradient descent and verify the $O(1/T^2)$ rate on a quadratic with $\kappa = 100$. Plot the suboptimality against T on a log-log scale and confirm the slope.
4. **(Medium)** Reproduce the [reddi2018convergence] counterexample: construct a simple one-dimensional convex problem on which Adam fails to converge under constant step size. Verify that AMSGrad converges.
5. **(Medium)** Implement SGD with mini-batching on a logistic regression problem. Plot the loss as a function of wall-clock time for batch sizes $\{1, 16, 256, 4096\}$. Identify the regime where larger batches help and the regime where they hurt.
6. **(Hard)** Prove the SGD-under-PL theorem: SGD under the PL condition with constant step size converges linearly to a noise floor $\sigma^2/(2\mu L)$.
7. **(Hard)** Show that, for a quadratic with Hessian $\mathbf{A}$, the convergence rate of gradient descent is determined by $\kappa = \lambda_{\max}/\lambda_{\min}$. Show that preconditioning by $\mathbf{A}^{-1}$ gives one-step convergence (and explain why this is not useful in practice).
8. **(Research)** Investigate the loss landscape of a small transformer trained on a simple algorithmic task (e.g., modular addition). How does the loss surface change with depth, width, and training data size? Are the minima connected?

Probability and Concentration

Probability enters deep learning in three distinct ways. As a **model**: a neural network's outputs are interpreted as parameters of a probability distribution over labels, and the loss is a (negative) log-likelihood. As a **tool**: stochastic gradient descent, mini-batching, dropout, and data augmentation are all probabilistic algorithms. As a **theoretical lens**: generalization is a statement about the gap between the empirical risk (a sample average) and the population risk (an expectation), and the size of this gap is controlled by concentration inequalities.

This chapter develops the probability required to read the modern generalization literature. The first three sections treat the basic inequalities (Markov, Chebyshev, Chernoff), sub-Gaussian and sub-exponential random variables, and McDiarmid's bounded-differences inequality. The fourth section treats uniform convergence — VC dimension and Rademacher complexity — which is the classical foundation of generalization theory. The fifth section explains why classical uniform convergence is *vacuous* for overparameterized deep networks, motivating the modern theory developed in Volume III.

4.1 Basic inequalities

The most basic concentration inequality is **Markov's**, which requires only non-negativity.

Theorem (Markov's inequality)

For any non-negative random variable X and $t > 0$,

$$\Pr[X \geq t] \leq \frac{\mathbb{E}[X]}{t}.$$

Markov is sharp in general but loose for specific distributions. The power of Markov is that it can be applied to powers or exponentials of X to obtain tighter bounds — this is the route to Chernoff bounds.

Theorem (Chebyshev's inequality)

For any random variable X with finite mean μ and variance σ^2 ,

$$\Pr[|X - \mu| \geq t] \leq \frac{\sigma^2}{t^2}.$$

Chebyshev is Markov applied to $(X - \mu)^2$. It already exhibits the key feature of concentration: deviations scale as $1/\sqrt{t}$ in probability, not as $1/t$. For

sums of n independent random variables, the variance grows linearly in n while the mean grows linearly in n , so deviations are of order $\sqrt{n}$ — the central limit theorem regime.

4.2 Sub-Gaussian and sub-exponential random variables

The right framework for sharp concentration is the **moment generating function**.

Definition (Sub-Gaussian random variable)

A random variable X with mean $\mu = \mathbb{E}[X]$ is **sub-Gaussian** with parameter σ if, for all $\lambda \in \mathbb{R}$,

$$\mathbb{E}[\exp(\lambda(X - \mu))] \leq \exp(\sigma^2 \lambda^2 / 2).$$

Gaussian random variables with variance σ^2 are sub-Gaussian with parameter σ ; bounded random variables (e.g., Rademacher ± 1) are sub-Gaussian with parameter depending on their range. The class of sub-Gaussian random variables is closed under summation and scaling, which makes it the right class for analyzing sums of independent random variables.

Theorem (Hoeffding's inequality)

Let $X_1, \dots, X_n$ be independent, mean-zero, sub-Gaussian random variables with parameter σ . Then for any $t > 0$,

$$\Pr\left[\left|\sum_{i=1}^n X_i\right| \geq t\right] \leq 2 \exp\left(-\frac{t^2}{2n\sigma^2}\right).$$

Hoeffding's inequality is the workhorse concentration result: the sum of n independent sub-Gaussian random variables is sub-Gaussian with parameter $\sigma\sqrt{n}$. Equivalently, the *average* $\frac{1}{n} \sum X_i$ is sub-Gaussian with parameter $\sigma/\sqrt{n}$, giving the canonical $1/\sqrt{n}$ scaling of statistical fluctuations.

For random variables with heavier tails — products of sub-Gaussians, for example — the right class is **sub-exponential**.

Definition (Sub-exponential random variable)

A mean-zero random variable X is **sub-exponential** with parameters (ν, b) if, for all $|\lambda| < 1/b$,

$$\mathbb{E}[\exp(\lambda X)] \leq \exp(\nu^2 \lambda^2 / 2).$$

The sum of n independent sub-exponential random variables is sub-exponential with parameters $(\nu\sqrt{n}, b)$, and the corresponding concentration inequality (Bernstein's) has two regimes: Gaussian-like $\exp(-t^2/(2n\nu^2))$ for small t , and exponential $\exp(-t/(2b))$ for large t .

4.3 McDiarmid's inequality

Many functions of interest — the loss of a trained network as a function of the training data, for example — are not sums of independent random variables. McDiarmid's inequality handles the more general case of **bounded-differences** functions.

Theorem (McDiarmid's inequality)

Let $X_1, \dots, X_n$ be independent random variables taking values in a set $\mathcal{X}$, and let $f : \mathcal{X}^n \rightarrow \mathbb{R}$ satisfy the **bounded differences** condition: there exist $c_1, \dots, c_n$ such that for all i and all $x_1, \dots, x_n, x'_i$,

$$|f(x_1, \dots, x_i, \dots, x_n) - f(x_1, \dots, x'_i, \dots, x_n)| \leq c_i.$$

Then for any $t > 0$,

$$\Pr[|f(X_1, \dots, X_n) - \mathbb{E}[f]| \geq t] \leq 2 \exp\left(-\frac{2t^2}{\sum_i c_i^2}\right).$$

McDiarmid's inequality is the master tool for proving concentration of functions of independent random variables. Hoeffding's inequality is the special case $f(x_1, \dots, x_n) = \sum_i x_i$. The empirical process inequalities of the next section are derived by applying McDiarmid to suprema of empirical processes.

4.4 Uniform convergence and classical learning theory

The generalization problem is to bound the gap between empirical risk $\hat{\mathcal{L}}(\theta) = \frac{1}{N} \sum_{i=1}^N \ell(\theta; \mathbf{z}_i)$ and population risk $\mathcal{L}(\theta) = \mathbb{E}[\ell(\theta; \mathbf{Z})]$. For a *fixed* θ , this gap is controlled by Hoeffding: $|\hat{\mathcal{L}}(\theta) - \mathcal{L}(\theta)| = O(\sqrt{1/N})$ with high probability. The challenge is that θ is *not* fixed — it is chosen by minimizing $\hat{\mathcal{L}}$ — and the bound must hold uniformly over the function class $\mathcal{F} = \{f_\theta : \theta \in \Theta\}$.

4.4.1 VC dimension and Rademacher complexity

Two complexity measures of function classes dominate the classical theory.

Definition (VC dimension)

The **Vapnik–Chervonenkis (VC) dimension** of a function class $\mathcal{F}$ of binary classifiers is the largest d such that there exists a set of d points $\{\mathbf{x}_1, \dots, \mathbf{x}_d\}$ that $\mathcal{F}$ can shatter — i.e., realize every labeling.

Definition (Rademacher complexity)

The **empirical Rademacher complexity** of a function class $\mathcal{F}$ on a sample $S = (\mathbf{z}_1, \dots, \mathbf{z}_N)$ is

$$\hat{\mathfrak{R}}_S(\mathcal{F}) = \mathbb{E}_\sigma \left[\sup_{f \in \mathcal{F}} \frac{1}{N} \sum_{i=1}^N \sigma_i f(\mathbf{z}_i) \right],$$

where σ_i are independent Rademacher (± 1) random variables. The (population) Rademacher complexity is $\mathfrak{R}_N(\mathcal{F}) = \mathbb{E}_S[\hat{\mathfrak{R}}_S(\mathcal{F})]$.

Theorem (Rademacher generalization bound)

With probability $1 - \delta$ over the sample S of size N , for all $f \in \mathcal{F}$,

$$\mathcal{L}(f) \leq \hat{\mathcal{L}}(f) + 2\mathfrak{R}_N(\mathcal{F}) + \sqrt{\frac{2 \log(2/\delta)}{N}}.$$

The Rademacher complexity is a data-dependent complexity measure — it depends on the distribution of $\mathbf{Z}$, not just on the function class. For neural networks with P parameters and Lipschitz activations, the Rademacher complexity is $O(\sqrt{P/N})$ in the worst case, leading to the classical wisdom that one needs $N \gg P$ samples to generalize.

4.4.2 Why classical bounds are vacuous for deep networks

Modern deep networks have $P \sim 10^9$ parameters and are trained on $N \sim 10^9$ examples; classical bounds based on parameter count give a generalization gap of order $O(1)$, which is useless. Worse, [zhang2021understanding] showed that networks can fit *randomly labeled* data perfectly — they have enough capacity to memorize the training set — yet generalize well on real data. The function class is the same in both cases; only the data differs. Classical uniform convergence cannot distinguish the two.

This is the entry point for the modern theory of generalization, which proceeds by exploiting the *implicit bias* of SGD (the algorithm selects specific minima from the many available), the *structure* of real data (which makes memorization unnecessary), and the *compressibility* of trained networks (which gives a smaller effective complexity than the parameter count suggests). We develop these ideas in Volume III, Chapter 4.

4.5 Computational experiments

4.5.1 Experiment 1: Hoeffding vs. Chebyshev

Sample n Bernoulli($1/2$) random variables for various n . Plot the empirical tail probability $\Pr[|\bar{X}_n - 1/2| \geq t]$ against the bounds given by Chebyshev and Hoeffding. Verify that Hoeffding is exponentially tighter.

4.5.2 Experiment 2: Rademacher complexity of a linear class

Compute the empirical Rademacher complexity of the class $f_w(\mathbf{x}) = \mathbf{w}^\top \mathbf{x}$ with $\|\mathbf{w}\| \leq B$ on a synthetic Gaussian dataset. Verify the theoretical bound $\hat{\mathfrak{R}}_S \leq B\|\bar{\mathbf{x}}\|/\sqrt{N}$.

4.6 Historical notes

The basic concentration inequalities (Markov, Chebyshev, Chernoff) are 19th- and mid-20th-century results, with Chernoff’s bound appearing in [chernoff1952measure]. Hoeffding’s inequality is due to [hoeffding1963probability]; the extension to sub-Gaussian random variables is developed systematically in [boucheron2013concentration]. McDiarmid’s inequality is due to [mcdiarmid1989method].

VC dimension was introduced by [vapnik1971uniform]. Rademacher complexity has a longer history in empirical process theory; its introduction to learning theory is due to [koltchinskii2001rademacher] and [bartlett2002rademacher]. The recognition that classical uniform convergence is vacuous for overparameterized networks is due to [zhang2021understanding], which launched the modern theory of generalization in deep learning.

4.7 Exercises

1. **(Easy)** Prove that a bounded random variable $X \in [a, b]$ is sub-Gaussian with parameter $(b - a)/2$.
2. **(Easy)** Show that a sum of independent sub-Gaussian random variables is sub-Gaussian, with parameter equal to the ℓ_2 norm of the individual parameters.
3. **(Medium)** Derive Hoeffding’s inequality from the sub-Gaussian definition. (Hint: apply Markov to $\exp(\lambda \sum X_i)$ and optimize over λ .)
4. **(Medium)** Compute the VC dimension of half-spaces in $\mathbb{R}^2$. Compute the VC dimension of the class of axis-aligned rectangles in $\mathbb{R}^d$.
5. **(Medium)** For a fixed function f with $\ell(f; \mathbf{z}) \in [0, 1]$, derive a high-probability bound on $|\hat{\mathcal{L}}(f) - \mathcal{L}(f)|$ as a function of N and the confidence δ .
6. **(Hard)** Prove the Rademacher generalization bound. (Hint: symmetrization, then McDiarmid.)
7. **(Hard)** Show that the Rademacher complexity of a class of neural networks with P parameters, L layers, and Lipschitz activations grows at most polynomially in P under suitable normalization. Compare this to the naive $O(\sqrt{P/N})$ bound.
8. **(Research)** Reproduce a variant of the [zhang2021understanding] experiment: train a small CNN on (a) CIFAR-10 with true labels and (b) CIFAR-10 with random labels. Report training and test accuracy in both cases. Discuss what this implies about the relevance of uniform convergence bounds to deep learning.

Information Theory

Information theory enters deep learning in three ways. As a **modeling tool**: variational autoencoders, the evidence lower bound, and the information bottleneck principle are all explicitly information-theoretic. As a **training objective**: cross-entropy, KL divergence, and mutual-information maximization are standard losses. As a **theoretical lens**: the representations learned by deep networks are increasingly analyzed in terms of their information content — mutual information with the input, with the label, with intermediate representations. This chapter develops the information theory required to read this literature.

5.1 Entropy and mutual information

Definition (Shannon entropy)

For a discrete random variable X with probability mass function $p(x)$, the **Shannon entropy** is

$$H(X) = - \sum_x p(x) \log p(x),$$

with the convention $0 \log 0 = 0$. The unit is bits when log is base 2, nats when log is natural.

Entropy measures the *uncertainty* in X . A fair coin has $H = 1$ bit; a biased coin with $p = 0.01$ has $H \approx 0.08$ bits. The maximum entropy for a discrete random variable on k values is $\log k$, attained by the uniform distribution.

For continuous random variables, the analog is **differential entropy**, $h(X) = - \int p(x) \log p(x) dx$. Differential entropy can be negative (it is for a uniform on $[0, 0.1]$, which has $h = -\log 10 < 0$), and lacks the clean operational meaning of discrete entropy. It is nonetheless useful as a calculational tool.

Definition (KL divergence)

The **Kullback–Leibler (KL) divergence** from P to Q is

$$D_{\text{KL}}(P\|Q) = \mathbb{E}_P \left[\log \frac{p(X)}{q(X)} \right] = \int p(x) \log \frac{p(x)}{q(x)} dx.$$

KL divergence is non-negative (Gibbs' inequality) and zero iff $P = Q$ almost everywhere. It is *not* symmetric — $D_{\text{KL}}(P\|Q) \neq D_{\text{KL}}(Q\|P)$ in general — and does not satisfy the triangle inequality. It is, however, the canonical measure of dissimilarity between distributions in machine learning, both because of its statistical interpretation (it governs the

asymptotic rate of distinguishability) and its computational convenience.

Definition (Mutual information)

The **mutual information** between two random variables X and Y is

$$I(X; Y) = D_{\text{KL}}(P_{X,Y} \| P_X \otimes P_Y) = \mathbb{E}_{X,Y} \left[\log \frac{p(X,Y)}{p(X)p(Y)} \right].$$

Mutual information is symmetric ($I(X; Y) = I(Y; X)$), non-negative, and zero iff X and Y are independent. It measures the *information* Y carries about X (or vice versa). For Gaussian (X, Y) with correlation ρ , $I(X; Y) = -\frac{1}{2} \log(1 - \rho^2)$.

5.2 The data processing inequality

Theorem (Data processing inequality)

If $X \rightarrow Y \rightarrow Z$ is a Markov chain (i.e., Z is conditionally independent of X given Y), then

$$I(X; Z) \leq I(X; Y).$$

The data processing inequality (DPI) formalizes the intuition that *post-processing cannot create information*: if Z is a (possibly randomized) function of Y alone, and Y is the only channel through which information about X flows, then Z cannot contain more information about X than Y does.

The DPI has a sharp consequence for deep networks: if we view the network's layers as a Markov chain $X \rightarrow h_1 \rightarrow h_2 \rightarrow \dots \rightarrow h_L \rightarrow \hat{Y}$, then $I(X; h_\ell)$ is *non-increasing* in ℓ . The intermediate representations cannot contain more information about the input than earlier layers do — a constraint that, combined with the goal of preserving information about the label Y , gives the information bottleneck principle.

5.3 The information bottleneck principle

[tishby2015deep] and [tishby1999information] proposed that deep learning can be understood as a process of *compressing* the input X while *preserving* information about the label Y . The objective is

$$\min_{p(h|x)} I(X; H) - \beta I(H; Y),$$

where $\beta > 0$ is a trade-off parameter. The first term pushes the representation H to compress X ; the second pushes it to retain information about Y . The Lagrangian reveals that the optimal representation is a *minimal sufficient statistic* for Y from X .

The empirical claim — controversial but influential — is that the layers of a trained network traverse a trajectory in the **information plane** ($I(X; h_\ell), I(h_\ell; Y)$), with an early *fitting* phase in which both quantities grow, and a later *compression* phase in which $I(X; h_\ell)$ decreases while $I(h_\ell; Y)$ stays roughly constant. The compression phase has been argued to underlie generalization. The empirical status of this claim is the subject of ongoing debate; see Volume III, Chapter 4.

5.4 Variational inference and the ELBO

Many probabilistic models in deep learning — variational autoencoders [kingma2013auto], deep latent Gaussian models — involve an intractable posterior $p(\mathbf{z}|\mathbf{x})$. Variational inference approximates the posterior by a tractable distribution $q_\phi(\mathbf{z}|\mathbf{x})$ by maximizing the **evidence lower bound (ELBO)**:

$$\log p(\mathbf{x}) \geq \mathbb{E}_{q_\phi(\mathbf{z}|\mathbf{x})}[\log p(\mathbf{x}|\mathbf{z})] - D_{\text{KL}}(q_\phi(\mathbf{z}|\mathbf{x})\|p(\mathbf{z})).$$

The ELBO is derived by applying Jensen’s inequality to $\log p(\mathbf{x}) = \log \int p(\mathbf{x}, \mathbf{z}) d\mathbf{z}$. The first term is the *reconstruction* term — it rewards the model for explaining the data — and the second is the *regularization* term — it penalizes the approximate posterior for straying from the prior. The two terms are in tension, controlled by the choice of prior and the capacity of q_ϕ .

The reparameterization trick [kingma2013auto] allows the ELBO to be optimized by stochastic gradient descent: write $\mathbf{z} = g_\phi(\mathbf{x}, \epsilon)$ for a deterministic function g_ϕ and a noise variable ϵ with a fixed distribution, then differentiate through g_ϕ . This turns an expectation over q_ϕ into an expectation over a fixed distribution, which can be estimated by Monte Carlo and differentiated.

5.5 Cross-entropy and softmax

The most common loss for classification is **cross-entropy**:

$$\ell_{\text{CE}}(\theta; \mathbf{x}, y) = -\log p_\theta(y|\mathbf{x}) = -\log \frac{\exp(f_y(\mathbf{x}; \theta))}{\sum_{y'} \exp(f_{y'}(\mathbf{x}; \theta))},$$

where $f_y(\mathbf{x}; \theta)$ is the logit for class y . The cross-entropy is the negative log-likelihood under a softmax distribution; equivalently, it is the KL divergence (up to a constant) between the true label distribution (a one-hot) and the predicted distribution. The gradient with respect to the logits has the clean form $\partial \ell / \partial f_y = p_\theta(y|\mathbf{x}) - \mathbf{1}[y = y']$.

The **temperature-scaled softmax** $p(y|\mathbf{x}) = \exp(f_y/T) / \sum_{y'} \exp(f_{y'}/T)$ interpolates between a uniform distribution (large T) and an argmax (small T). The temperature plays a central role in knowledge distillation: a smaller student network is trained to match the temperature-scaled outputs of a larger teacher, transferring not just the hard label but the teacher’s *soft* distribution over classes.

5.6 Computational experiments

5.6.1 Experiment 1: Mutual information estimation

Estimating mutual information from samples is hard in general; the naive plug-in estimator is biased and high-variance. The following code implements the **MINE** estimator (Mutual Information Neural Estimation, [belghazi2018mine]), which uses a neural network to lower-bound $I(X; Y)$ via the Donsker–Varadhan representation.

```
import torch
import torch.nn as nn

class MINE(nn.Module):
    """Mutual Information Neural Estimator.
    Lower-bounds I(X;Y) via Donsker-Varadhan:
    I(X;Y) >= E_{P_{XY}}[T(x,y)] - log E_{P_X}
    ↪ P_Y}[exp(T(x,y))]
    """
    def __init__(self, dim_x, dim_y, hidden=64):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(dim_x + dim_y, hidden),
            ↪ nn.ReLU(),
            nn.Linear(hidden, hidden), nn.ReLU(),
            nn.Linear(hidden, 1)
        )
    def forward(self, x, y):
        # joint samples
        t_joint = self.net(torch.cat([x, y],
        ↪ dim=1)).mean()
        # marginal samples: shuffle y to break
        ↪ dependence
        y_marg = y[torch.randperm(y.shape[0])]
```

```

t_marg = torch.logsumexp(
    self.net(torch.cat([x, y_marg],
        ↪ dim=1)).squeeze(), dim=0
    ) -
↪ torch.log(torch.tensor(float(x.shape[0])))
return t_joint - t_marg # lower bound on
↪ I(X;Y)

```

5.6.2 Experiment 2: ELBO and reconstruction

Implement a small VAE on MNIST. Plot the reconstruction term, the KL term, and the ELBO as a function of training epoch. Observe that the KL term grows quickly early in training (the posterior learns to differ from the prior) and then plateaus, while the reconstruction term continues to decrease.

5.7 Historical notes

Information theory was founded by [shannon1948mathematical], who introduced entropy, mutual information, and the basic coding theorems. The data processing inequality appears in its modern form in [cover1999elements]. The Kullback–Leibler divergence is due to [kullback1951information].

The information bottleneck principle was introduced by [tishby1999information] and applied to deep learning in [tishby2015deep]. The empirical study of information flow in deep networks has been controversial; [saxe2018on] showed that the compression phase depends critically on the choice of activation function (saturating vs. non-saturating) and on the mutual information estimator. The current consensus is that the IB principle is a useful *perspective* on representation learning but not a complete *theory*.

Variational inference has a long history in Bayesian statistics; the modern formulation in terms of the ELBO is due to [jordan1999introduction]. The variational autoencoder was introduced in [kingma2013auto] and [rezende2014stochastic]; the reparameterization trick is the key technical innovation that makes the ELBO differentiable. Knowledge distillation is due to [hinton2015distilling].

5.8 Exercises

1. **(Easy)** Show that $D_{\text{KL}}(P\|Q) \geq 0$ with equality iff $P = Q$. (Hint: use Jensen’s inequality on $-\log$.)
2. **(Easy)** Compute the entropy of a Bernoulli(p) random variable. Plot it as a function of p and verify that the maximum is at $p = 1/2$.
3. **(Medium)** Prove the data processing inequality. (Hint: use the chain rule for mutual information and the conditional independence.)
4. **(Medium)** For a bivariate Gaussian (X, Y) with correlation ρ , compute $I(X; Y)$ in closed form. Verify numerically that the MINE estimator recovers this value to within estimator error.
5. **(Medium)** Derive the ELBO from Jensen’s inequality. Show that the gap $\log p(\mathbf{x}) - \text{ELBO}$ is exactly $D_{\text{KL}}(q_\phi(\mathbf{z}|\mathbf{x})\|p(\mathbf{z}|\mathbf{x}))$.
6. **(Hard)** Implement a small VAE on MNIST. Train it and plot reconstructions and samples from the prior. Vary the dimension of the latent space and observe the trade-off between reconstruction quality and the KL term.
7. **(Hard)** Implement the information bottleneck objective for a small classifier on a synthetic dataset with a known relevant subspace. Verify that the learned representation concentrates information about the label and discards information about the rest of the input.
8. **(Research)** Reproduce the [tishby2015deep] experiment on the information plane trajectory of a small network. Use the MINE estimator (or a comparable one) and plot $I(X; h_\ell)$ and $I(h_\ell; Y)$ as a function of training epoch. Does compression occur? Does it depend on the activation function?

Part II.

Algorithms and Architectures

Introduction to Volume II

Volume II treats the principal architectures of modern deep learning. Each chapter follows a uniform structure: the architecture is introduced as a function class with a specific inductive bias; the relevant theorems (expressivity, universal approximation, equivariance) are stated and proved or cited; the training algorithm is given explicitly in pseudocode and Python; the empirical performance is reported with references to the primary literature; and the historical notes situate the architecture in the trajectory of the field.

The unifying perspective is that **architectures are constraints on the function class**. Convolution imposes translation equivariance. Recurrence imposes temporal consistency. Attention imposes permutation equivariance. Residual connections impose a bias toward the identity and away from zero. Each constraint makes some functions easy to express and others hard; the choice of architecture is the choice of which constraints match the structure of the problem.

6.1 Chapter map

Chapter 5 — Multilayer Perceptrons. The simplest deep architecture: alternating affine maps and elementwise nonlinearities. Universal approximation (Cybenko, Hornik), the depth-vs-width tradeoff, the role of activation functions, and the practical considerations of initialization, normalization, and regularization.

Chapter 6 — Convolutional Networks. The architecture that ignited the deep learning era. Translation equivariance, the convolution-as-multiplication view, pooling and invariance, the canonical architectures (LeNet, AlexNet, VGG, ResNet, EfficientNet), and the modern successors (vision transformers, convnext).

Chapter 7 — Recurrent Networks. Sequence models before attention. RNNs, LSTMs, GRUs, the vanishing and exploding gradient problem, the connection to dynamical systems, and the practical considerations of sequence length and training stability.

Chapter 8 — The Transformer. The architecture that has come to dominate natural language processing and increasingly vision, speech, and other modalities. Self-attention, multi-head atten-

tion, positional encodings, the canonical architectures (BERT, GPT, T5), and the scaling properties that have driven the field since 2017.

Chapter 9 — Generative Models. The three principal families: variational autoencoders, generative adversarial networks, and diffusion models. Each is treated as a different solution to the problem of fitting an implicit or explicit density model to data.

Chapter 10 — The Practice of Training. The engineering of large-scale training: initialization, normalization, regularization, optimization, hyperparameter tuning, distributed training, and the monitoring of training runs. This is the chapter a practitioner needs; the others supply the conceptual context.

6.2 How to read

Chapter 5 is a prerequisite for the others; the remaining chapters can be read in any order. The computational experiments in each chapter are runnable on a single modern GPU; the exercises range from routine to research-level.

Multilayer Perceptrons

The multilayer perceptron (MLP) — also called a feedforward network — is the simplest deep architecture. It is a composition of affine maps and elementwise nonlinearities:

$$f_{\theta}(\mathbf{x}) = (\mathbf{W}_L \sigma(\cdot) + \mathbf{b}_L) \circ \dots \circ (\mathbf{W}_1 \mathbf{x} + \mathbf{b}_1),$$

where σ is a fixed nonlinearity (ReLU, tanh, GELU, etc.) applied elementwise. Despite its simplicity, the MLP is the right starting point: it is the architecture in which the central questions of deep learning — expressivity, trainability, generalization — can be posed most cleanly, and the answers obtained for MLPs often transfer (with modification) to more complex architectures.

7.1 Universal approximation

The foundational theoretical result is that MLPs with a single hidden layer can approximate any continuous function on a compact set, given enough width.

Theorem (Cybenko–Hornik universal approximation theorem)

Let $\sigma : \mathbb{R} \rightarrow \mathbb{R}$ be a continuous, non-constant, bounded activation function (a “sigmoidal” activation). Let $K \subset \mathbb{R}^d$ be compact. Then for any continuous $f : K \rightarrow \mathbb{R}$ and any $\varepsilon > 0$, there exists an integer n and parameters $\{\alpha_i, \mathbf{w}_i, b_i\}_{i=1}^n$ such that the single-hidden-layer network

$$g(\mathbf{x}) = \sum_{i=1}^n \alpha_i \sigma(\mathbf{w}_i^\top \mathbf{x} + b_i)$$

satisfies $\sup_{\mathbf{x} \in K} |g(\mathbf{x}) - f(\mathbf{x})| < \varepsilon$.

The theorem was proved independently by [cybenko1989approximation] (for sigmoidal activations) and [hornik1989multilayer] (for general non-polynomial activations). The proof is non-constructive: it shows *existence* of suitable parameters, not how to find them by gradient descent. The gap between existence and learnability is the central theoretical puzzle of deep learning.

7.1.1 Depth vs. width

A striking refinement shows that **depth can be exponentially more efficient than width** for representing certain functions. [telgarsky2016benefits] constructed a family of functions on $[0, 1]$ that can be

represented by a deep ReLU network of size $O(k^2)$ but require width $\Omega(2^k)$ for any shallow network with the same accuracy. The intuition is that each layer of ReLU composes the function with itself, allowing deep networks to express oscillatory functions exponentially more compactly.

Theorem (Telgarsky’s depth separation)

For each k , there exists a function $f_k : [0, 1] \rightarrow \{0, 1\}$ that can be computed by a ReLU network of depth k and $O(k^2)$ parameters, but any ReLU network of depth $k' < k$ computing f_k to accuracy $1/2 - \varepsilon$ requires width $\Omega(2^{k-k'})$.

This is a worst-case separation; whether depth provides similar advantages for the functions that arise in practice is an empirical question. The empirical record — ResNets with hundreds of layers outperform shallow networks on ImageNet, deep transformers outperform shallow ones on language tasks — suggests that it does, for natural data.

7.2 Activation functions

The choice of activation function is consequential. We summarize the principal options.

The Rectified Linear Unit (ReLU) [nair2010rectified] was the dominant activation in the early deep learning era; it addresses the vanishing gradient problem of saturating activations like sigmoid and tanh. GELU [hendrycks2016gaussian] has become the default in transformer architectures since BERT, partly for empirical reasons (better performance) and partly for theoretical reasons (a smoother profile that better matches the statistics of natural data).

7.3 Initialization

The choice of initialization is one of the most consequential design decisions for training deep networks. Poor initialization leads to vanishing or exploding activations and gradients, especially in deep networks.

Definition (Xavier/Glorot initialization)

For a layer with d_{in} inputs and d_{out} outputs, initialize weights as $W_{ij} \sim \mathcal{N}(0, 2/(d_{\text{in}} + d_{\text{out}}))$ or uniform on $[-\sqrt{6/(d_{\text{in}} + d_{\text{out}})}, \sqrt{6/(d_{\text{in}} + d_{\text{out}})}]$.

Activation	Formula	Properties
ReLU	$\max(0, x)$	Non-saturating for $x > 0$; sparse activations; “dead neuron” problem for $x < 0$
Leaky ReLU	$\max(\alpha x, x)$, $\alpha \ll 1$	Addresses dead neurons; sparse activations lost
ELU	x if $x > 0$, $\alpha(e^x - 1)$ else	Smooth; zero-mean outputs; saturating for $x \rightarrow -\infty$
GELU	$x\Phi(x)$, $\Phi =$ Gaussian CDF	Smooth; used in transformers; non-saturating
SwiGLU	$\text{Swish}(xW_1) \otimes (xW_2)$	Gated; used in modern transformers
Tanh	$\tanh(x)$	Bounded, smooth; saturating; classical

Xavier initialization [glorot2010understanding] is derived by requiring that the variance of activations and gradients is preserved across layers, under the assumption of linear activations. For ReLU activations, which zero out half of the inputs, the appropriate scaling is doubled:

Definition (He/Kaiming initialization)

For a layer with d_{in} inputs followed by a ReLU, initialize weights as $W_{ij} \sim \mathcal{N}(0, 2/d_{\text{in}})$.

He initialization [he2015delving] is the standard for ReLU-based networks. For other activations, the appropriate scaling can be derived analytically or tuned empirically.

7.4 Normalization

Even with careful initialization, the statistics of activations can drift during training, especially in deep networks. Normalization layers stabilize training by re-centering and re-scaling activations.

Definition (Batch normalization)

For a mini-batch $B = \{\mathbf{x}_1, \dots, \mathbf{x}_m\}$, batch normalization computes

$$\hat{\mathbf{x}}_i = \gamma \odot \frac{\mathbf{x}_i - \mu_B}{\sqrt{\sigma_B^2 + \varepsilon}} + \beta,$$

where μ_B, σ_B^2 are the batch mean and variance, γ, β are learned, and ε is for numerical stability.

Batch normalization [ioffe2015batch] was a key enabler of very deep networks; it allows higher learning rates, reduces sensitivity to initialization, and provides a mild regularizing effect (the batch statistics are noisy). Its dependence on batch size is a drawback for small batches and for distributed training; **layer normalization** [ba2016layer], which normalizes over features rather than batch elements, avoids this and is the default in transformers.

7.5 Dropout

Dropout [srivastava2014dropout]; [hinton2012improving] is a regularization technique that randomly zeroes a fraction p of activations during training, then scales by $1/(1 - p)$ at test time. The effect is to approximate training an ensemble of subnetworks (one per dropout mask)

and averaging their predictions. Dropout is theoretically motivated as preventing *co-adaptation* of features: each feature must be useful on its own, not in combination with specific other features.

7.6 A complete MLP in PyTorch

```
import torch
import torch.nn as nn

class MLP(nn.Module):
    def __init__(self, dims, activation=nn.GELU,
        ↪ p_drop=0.1):
        """dims: list like [784, 256, 64, 10]"""
        super().__init__()
        layers = []
        for i in range(len(dims) - 1):
            layers.append(nn.Linear(dims[i],
        ↪ dims[i+1]))
            if i < len(dims) - 2: # no norm/act on
        ↪ final layer
            layers.append(nn.BatchNorm1d(dims[i+1]))
            layers.append(activation())
            layers.append(nn.Dropout(p_drop))
        self.net = nn.Sequential(*layers)
        self._init_weights()
    def _init_weights(self):
        for m in self.net:
            if isinstance(m, nn.Linear):
                nn.init.kaiming_normal_(m.weight,
        ↪ nonlinearity='relu')
                nn.init.zeros_(m.bias)
    def forward(self, x):
        return self.net(x.flatten(1))

model = MLP([784, 256, 64, 10])
```

7.7 Computational experiments

7.7.1 Experiment 1: Universal approximation on 1D

Train a single-hidden-layer ReLU MLP to approximate a 1D function (e.g., $\sin(2\pi x) + 0.3\sin(8\pi x)$) to high accuracy. Plot the learned function and the network’s approximation as a function of the number of hidden units.

7.7.2 Experiment 2: Depth vs. width on a classification task

On MNIST, compare a shallow network (1 hidden layer of width 1024) to a deep network (4 hidden layers of width 256). Both have approximately the same parameter count. Plot training and test accuracy as a function of training epoch.

7.7.3 Experiment 3: Effect of initialization

Train the MLP from Listing the listing above on MNIST with three initializations: (a) all weights = 0.01, (b) Xavier, (c) He. Plot the loss curves and the activation statistics per layer.

7.8 Historical notes

The perceptron, the single-layer predecessor of the MLP, was introduced by [rosenblatt1958perceptron]. The limitations of single-layer perceptrons (their inability to learn XOR) were emphasized by [minsky2017perceptron], leading to the “AI winter” of the 1970s. The backpropagation algorithm, which made training multi-layer networks tractable, was popularized by [rumelhart1986learning], though it had been described earlier by Werbos (1974) and others.

Universal approximation was proved independently by [cybenko1989approximation] and [hornik1989multilayer]. The depth-separation results are more recent; [telgarsky2016benefits] gave the first clean separation in the ReLU case. ReLU was introduced to deep learning by [nair2010rectified], building on earlier work in computational neuroscience. Xavier initialization is due to [glorot2010understanding]; He initialization to [he2015delving]. Batch normalization is due to [ioffe2015batch]; layer normalization to [ba2016layer]. Dropout is due to [srivastava2014dropout] and [hinton2012improving].

7.9 Exercises

1. **(Easy)** Show that a single-hidden-layer network with ReLU activations is a piecewise-linear function of the input. How many “pieces” can a network with n hidden units have?
2. **(Easy)** Derive the gradient of the loss with respect to $\mathbf{W}_1, \mathbf{b}_1, \mathbf{W}_2, \mathbf{b}_2$ for a two-layer MLP with squared loss.
3. **(Medium)** Prove the universal approximation theorem for sigmoidal activations. (Hint: use the density of step functions in L^1 and the ability of sigmoids to approximate steps.)
4. **(Medium)** Implement Xavier and He initialization. Train a 10-layer MLP on MNIST with each,

plus a “bad” initialization (e.g., all weights = 1). Report the activation and gradient norms per layer.

5. **(Medium)** Implement batch normalization as a custom layer. Compare training curves with and without batch norm on a deep MLP (10+ layers).
6. **(Hard)** Reproduce a depth-separation experiment: construct a function that is easy for a deep network to represent and hard for a shallow one. Train both and compare sample efficiency.
7. **(Hard)** Show that dropout with rate p is equivalent, in expectation, to training an ensemble of 2^n subnetworks (where n is the number of dropout units) and averaging. Why is the explicit ensemble infeasible?
8. **(Research)** Investigate the role of activation function choice on the neural tangent kernel. How does the NTK change when the activation is ReLU vs. GELU vs. tanh? What are the implications for training dynamics?

Convolutional Networks

The convolutional neural network (CNN) is the architecture that ignited the modern deep learning era. By imposing **translation equivariance** through weight sharing and local connectivity, CNNs achieve dramatically better sample efficiency than MLPs on data with spatial structure — images, audio spectrograms, and (with modification) time series. The success of AlexNet [krizhevsky2012imagenet] on ImageNet in 2012 is conventionally dated as the start of the deep learning era; that success was, in retrospect, the convergence of convolutional architectures (known since [lecun1998gradient]’s LeNet), GPU computation, and large labeled datasets.

8.1 Translation equivariance

The defining structural property of a CNN is **translation equivariance**: shifting the input and applying the convolution gives the same result as applying the convolution and then shifting. Formally, if T_τ is the shift-by- τ operator, then $T_\tau \circ \text{conv}_w = \text{conv}_w \circ T_\tau$.

This equivariance is a property of the convolutional structure, not of the parameters. By using the same kernel w at every spatial location, the network learns features that detect the same pattern wherever it appears in the input. The savings in parameters, compared to a fully connected layer, is dramatic: a 5×5 kernel on a 224×224 image has 25 parameters per output channel, versus $224^2 \approx 50000$ for a fully connected layer.

For tasks where the *location* of a pattern is irrelevant (image classification), translation equivariance is combined with **pooling** to obtain translation *invariance*. For tasks where location matters (object detection, segmentation), the equivariant representation is preserved through the network.

8.2 The convolution operation

A two-dimensional convolution (technically, a cross-correlation; the machine learning convention differs from the signal processing convention by a sign flip) of input $\mathbf{X} \in \mathbb{R}^{C_{\text{in}} \times H \times W}$ with kernel $\mathbf{W} \in \mathbb{R}^{C_{\text{out}} \times C_{\text{in}} \times k \times k}$ produces output $\mathbf{Y} \in \mathbb{R}^{C_{\text{out}} \times H' \times W'}$ with

$$Y_{c',i,j} = \sum_{c=1}^{C_{\text{in}}} \sum_{u=1}^k \sum_{v=1}^k W_{c',c,u,v} X_{c,i+u-\lfloor k/2 \rfloor, j+v-\lfloor k/2 \rfloor} + b_{c'}.$$

The hyperparameters are: kernel size k , stride s (step size of the kernel), padding p (number of zeros added at the boundary), and dilation d (spacing between kernel elements). The output spatial dimensions are $H' = \lfloor (H + 2p - d(k - 1) - 1)/s \rfloor + 1$.

The computational cost of a convolutional layer is $O(C_{\text{out}} C_{\text{in}} k^2 H' W')$. The dominant cost in modern CNNs is usually the $C_{\text{out}} C_{\text{in}}$ factor; the spatial dimensions $H' W'$ shrink through the network via pooling and strided convolutions.

8.3 Canonical architectures

The evolution of CNN architectures tracks the evolution of the field.

LeNet [lecun1998gradient] (1998). The original CNN for digit recognition: two convolutional layers followed by fully connected layers. Total parameters: $\sim 60,000$.

AlexNet [krizhevsky2012imagenet] (2012). The first large-scale CNN trained on GPUs: 5 convolutional layers, 3 fully connected, ReLU activations, dropout, $\sim 60\text{M}$ parameters. Won the ImageNet 2012 competition by a wide margin and triggered the deep learning era.

VGG [simonyan2014very] (2014). Stacked 3×3 convolutions, demonstrating that depth alone (with small kernels) improves performance. $\sim 138\text{M}$ parameters; very heavy.

ResNet [he2015deep] (2015). Introduced **residual connections** $\mathbf{y} = \mathbf{x} + F(\mathbf{x})$, which bypass the layer’s transformation. This addressed the degradation problem (deeper networks were harder to train, paradoxically) and enabled networks of hundreds or thousands of layers. ResNet-50 ($\sim 25\text{M}$ parameters) remains a workhorse backbone.

EfficientNet [tan2019efficientnet] (2019). Systematic compound scaling of width, depth, and resolution. Reached state-of-the-art accuracy with an order of magnitude fewer FLOPs than previous architectures.

ConvNeXt [liu2022convnet] (2022). Modernized ResNet design with lessons from vision transform-

ers: large kernels (7×7), inverted bottlenecks, layer normalization, GELU activations. Demonstrated that pure convolutional architectures remain competitive with attention-based ones.

```
import torch
import torch.nn as nn

class ResidualBlock(nn.Module):
    def __init__(self, channels, stride=1):
        super().__init__()
        self.conv1 = nn.Conv2d(channels, channels, 3,
            ↪ stride=stride, padding=1, bias=False)
        self.bn1 = nn.BatchNorm2d(channels)
        self.conv2 = nn.Conv2d(channels, channels, 3,
            ↪ stride=1, padding=1, bias=False)
        self.bn2 = nn.BatchNorm2d(channels)
        self.act = nn.GELU()
        self.downsample = nn.Identity() if stride ==
            ↪ 1 else nn.Sequential(
            ↪     nn.Conv2d(channels, channels, 1,
            ↪     ↪ stride=stride, bias=False),
            ↪     nn.BatchNorm2d(channels)
            ↪ )
    def forward(self, x):
        identity = self.downsample(x)
        out = self.act(self.bn1(self.conv1(x)))
        out = self.bn2(self.conv2(out))
        return self.act(out + identity)
```

8.4 Pooling and invariance

Pooling reduces the spatial resolution of the feature map, providing a degree of translation invariance and reducing the computational cost of subsequent layers. Max pooling takes the maximum over a local window; average pooling takes the average. **Global average pooling** averages over the entire spatial extent, producing a single value per channel; it is the standard way to transition from convolutional features to a fixed-length representation for classification.

Modern architectures often replace explicit pooling with **strided convolutions**, which learn the downsampling rather than fixing it. ConvNeXt, for instance, uses strided convolutions throughout.

8.5 Receptive field

The **receptive field** of a unit in a CNN is the region of the input that can affect its value. For a stack of L convolutional layers with kernel size k and stride 1, the receptive field is $1 + L(k - 1)$. With stride $s > 1$, the receptive field grows multiplicatively rather than additively.

The receptive field has implications for what the network can compute: a unit with a 5×5 receptive

field cannot detect patterns larger than 5×5 pixels. Architectures for high-resolution images (semantic segmentation, object detection) typically include dilated convolutions or attention layers to expand the receptive field without increasing parameter count.

8.6 Computational experiments

8.6.1 Experiment 1: Equivariance check

Construct a small CNN. Feed an image and a shifted version of the same image. Verify numerically that the output shifts correspondingly.

8.6.2 Experiment 2: Receptive field growth

For ResNet-18, compute the theoretical receptive field of each layer's output. Compare to the empirical receptive field (measured by the gradient of a unit's activation with respect to the input).

8.6.3 Experiment 3: Modern vs. classical

On CIFAR-10, compare LeNet, VGG-11, ResNet-18, and ConvNeXt-tiny. Plot accuracy, parameter count, and FLOPs. Discuss the trade-offs.

8.7 Historical notes

The convolutional architecture was inspired by the architecture of the visual cortex, as described by [hubel1962receptive]. The Neocognitron of [fukushima1982neocognitron] was an early biologically-inspired model with convolution-like weight sharing. LeNet [lecun1998gradient] was the first modern CNN, used commercially for digit recognition. AlexNet [krizhevsky2012imagenet] launched the deep learning era by demonstrating dramatic improvements on ImageNet. ResNet [he2015deep] enabled very deep networks through residual connections. Vision transformers [dosovitskiy2020image] challenged the dominance of CNNs on vision tasks in 2020; ConvNeXt [liu2022convnet] demonstrated that pure convolutional architectures remain competitive when modernized.

8.8 Exercises

1. (Easy) Show that a 3×3 convolution followed by another 3×3 convolution has the same recep-

tive field as a single 5×5 convolution. Compare parameter counts.

2. **(Easy)** Compute the receptive field of a unit in the final layer of a 4-layer CNN with 3×3 kernels and stride 1.
3. **(Medium)** Implement a dilated convolution. Show empirically that dilated convolutions can expand the receptive field without increasing parameter count.
4. **(Medium)** Implement and train a small ResNet on CIFAR-10. Compare training curves with and without residual connections.
5. **(Medium)** Implement depthwise-separable convolutions (used in MobileNet). Compare parameter count and accuracy to a standard convolution.
6. **(Hard)** Prove that a convolutional layer is translation equivariant. Under what conditions is it translation invariant?
7. **(Hard)** Implement a vision transformer (ViT) and compare its performance to a ResNet on CIFAR-10. How do they scale with dataset size?
8. **(Research)** Investigate the role of the kernel size in modern CNNs. Why did ConvNeXt choose 7×7 kernels? Are larger kernels (e.g., 31×31) ever beneficial?

Recurrent Networks

A recurrent neural network (RNN) processes a sequence $\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_T$ by maintaining a hidden state $\mathbf{h}_t$ that is updated at each step:

$$\mathbf{h}_t = \sigma(\mathbf{W}_h \mathbf{h}_{t-1} + \mathbf{W}_x \mathbf{x}_t + \mathbf{b}).$$

The same weights $\mathbf{W}_h, \mathbf{W}_x$ are used at every timestep — the network is **weight-shared across time** in the same way that a CNN is weight-shared across space. This weight sharing gives the RNN two key properties: (i) it can process sequences of arbitrary length, and (ii) the number of parameters is independent of the sequence length.

9.1 The vanishing and exploding gradient problem

The fundamental difficulty of training RNNs is the **vanishing and exploding gradient problem**. The gradient of the loss with respect to $\mathbf{h}_t$ involves a product of Jacobians:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{h}_t} = \frac{\partial \mathcal{L}}{\partial \mathbf{h}_T} \prod_{k=t+1}^T \frac{\partial \mathbf{h}_k}{\partial \mathbf{h}_{k-1}}.$$

If the largest eigenvalue of each Jacobian is less than 1, the product vanishes exponentially in $T - t$; if it is greater than 1, the product explodes. The Jacobian’s eigenvalues are bounded by the operator norm of $\mathbf{W}_h$ times the Lipschitz constant of σ ; for sigmoid or tanh activations, this is at most $\|\mathbf{W}_h\|$, so initialization must be carefully tuned to keep $\|\mathbf{W}_h\|$ near 1.

Theorem (RNN gradient bounds)

Consider a linear RNN $\mathbf{h}_t = \mathbf{W} \mathbf{h}_{t-1} + \mathbf{U} \mathbf{x}_t$ with $\|\mathbf{W}\| = \rho$. Then $\|\partial \mathcal{L} / \partial \mathbf{h}_t\| \leq \rho^{T-t} \|\partial \mathcal{L} / \partial \mathbf{h}_T\|$. The gradient vanishes if $\rho < 1$ and explodes if $\rho > 1$.

9.2 LSTM and GRU

The **Long Short-Term Memory** (LSTM) network [hochreiter1997long] addresses the vanishing gradient problem by introducing a **cell state** $\mathbf{c}_t$ that is updated additively rather than multiplicatively, and three **gates** (input, forget, output) that control the flow of information:

$$\begin{aligned} \mathbf{f}_t &= \sigma(\mathbf{W}_f[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_f) && \text{(forget gate)} \\ \mathbf{i}_t &= \sigma(\mathbf{W}_i[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_i) && \text{(input gate)} \\ \tilde{\mathbf{c}}_t &= \tanh(\mathbf{W}_c[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_c) && \text{(candidate)} \\ \mathbf{c}_t &= \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{c}}_t && \text{(cell update)} \\ \mathbf{o}_t &= \sigma(\mathbf{W}_o[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_o) && \text{(output gate)} \\ \mathbf{h}_t &= \mathbf{o}_t \odot \tanh(\mathbf{c}_t) && \text{(hidden state)} \end{aligned}$$

The forget gate $\mathbf{f}_t$ controls what is retained from the previous cell state; the input gate $\mathbf{i}_t$ controls what is written; the output gate $\mathbf{o}_t$ controls what is read out. The additive cell update is the key: the gradient flows through the cell state without multiplication by a Jacobian, addressing the vanishing gradient problem (at least in the limit of forget gate = 1).

The **Gated Recurrent Unit** (GRU) [cho2014learning] is a simplified variant with two gates (update, reset) and no separate cell state:

$$\begin{aligned} \mathbf{z}_t &= \sigma(\mathbf{W}_z[\mathbf{h}_{t-1}, \mathbf{x}_t]) && \text{(update gate)} \\ \mathbf{r}_t &= \sigma(\mathbf{W}_r[\mathbf{h}_{t-1}, \mathbf{x}_t]) && \text{(reset gate)} \\ \tilde{\mathbf{h}}_t &= \tanh(\mathbf{W}_h[\mathbf{r}_t \odot \mathbf{h}_{t-1}, \mathbf{x}_t]) \\ \mathbf{h}_t &= (1 - \mathbf{z}_t) \odot \mathbf{h}_{t-1} + \mathbf{z}_t \odot \tilde{\mathbf{h}}_t. \end{aligned}$$

GRU has fewer parameters than LSTM and is often competitive in practice.

9.3 Connection to dynamical systems

A continuous-time RNN is a dynamical system $\dot{\mathbf{h}}(t) = -\mathbf{h}(t) + \sigma(\mathbf{W}\mathbf{h}(t) + \mathbf{U}\mathbf{x}(t))$, discretized by an Euler scheme with step 1. This perspective connects RNNs to the rich theory of dynamical systems: fixed points, limit cycles, bifurcations, and chaos all have analogs in the behavior of trained RNNs. [sussillo2013opening] exploited this perspective to stabilize RNN training by initializing near the “edge of chaos.”

The **echo state property** [jaeger2001echo] characterizes when an untrained random RNN (with only the readout trained) can approximate any sequence-to-sequence map: roughly, the network must be contracting but not too contracting, with spectral radius of $\mathbf{W}_h$ near 1.

9.4 Computational experiments

9.4.1 Experiment 1: Vanishing gradient demonstration

Train a vanilla RNN and an LSTM on a synthetic task requiring long-range dependencies (e.g., “copy the first token of a length-50 sequence to the end”). Plot the gradient norm at the first timestep as a function of sequence length. The vanilla RNN’s gradient should vanish; the LSTM’s should not.

9.4.2 Experiment 2: LSTM on character-level language modeling

Train an LSTM on a small text dataset (e.g., Shakespeare). Plot training and validation loss as a function of epoch and sequence length. Generate samples from the trained model.

9.4.3 Experiment 3: GRU vs. LSTM

On a sequence classification task (e.g., MNIST as a 784-step sequence), compare LSTM and GRU in terms of accuracy, training time, and parameter count.

9.5 Historical notes

The RNN has its origins in the work of [rumelhart1986learning] on backpropagation through time. The vanishing gradient problem was identified by [hochreiter1991untersuchungen], leading directly to the LSTM [hochreiter1997long]. The GRU was introduced by [cho2014learning] as a simplified alternative. The connection to dynamical systems has been explored by [sussillo2013opening] and [poole2016exponential], among others.

The dominance of recurrent networks in sequence modeling ended with the introduction of the transformer [vaswani2017attention] in 2017, which we treat in the next chapter. RNNs remain useful in low-resource settings, in real-time streaming applications, and as a baseline against which newer architectures are compared.

9.6 Exercises

1. (Easy) Derive backpropagation through time (BPTT) for a vanilla RNN with one hidden layer.
2. (Easy) Show that the LSTM’s cell state gradient does not suffer from the vanishing gradient problem in the limit of forget gate = 1.
3. (Medium) Implement a vanilla RNN and an LSTM from scratch (no PyTorch RNN modules). Compare their behavior on a long-range dependency task.
4. (Medium) Implement gradient clipping. Show empirically that it stabilizes RNN training.
5. (Medium) Implement bidirectional RNNs and apply to a sequence labeling task (e.g., part-of-speech tagging).
6. (Hard) Reproduce the vanishing gradient experiment of [pascanu2013difficulty]: train RNNs of various spectral radii and measure the gradient norm at the first timestep.
7. (Hard) Implement an echo state network (random RNN, trained readout). What tasks can it solve? What are its limitations?
8. (Research) Investigate modern recurrent architectures (e.g., Mamba, RWKV) that aim to combine the benefits of RNNs (constant memory) with the performance of transformers. What inductive biases do they impose?

The Transformer

The transformer [vaswani2017attention] is the architecture that has come to dominate natural language processing and, increasingly, computer vision, speech, and other domains. Its central innovation is the **self-attention** mechanism, which replaces the recurrence of RNNs with a single, parallelizable operation that allows each position in a sequence to attend to all other positions. The combination of attention, residual connections, layer normalization, and positional encodings has proved remarkably scalable: transformers have been trained at scales from millions to hundreds of billions of parameters, with consistent improvements in capability.

10.1 Self-attention

Given a sequence of input vectors $\mathbf{X} \in \mathbb{R}^{n \times d}$, self-attention computes three projections — queries $\mathbf{Q} = \mathbf{X}\mathbf{W}_Q$, keys $\mathbf{K} = \mathbf{X}\mathbf{W}_K$, values $\mathbf{V} = \mathbf{X}\mathbf{W}_V$, all in $\mathbb{R}^{n \times d_h}$ — and outputs

$$\text{Attn}(\mathbf{X}) = \text{softmax} \left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_h}} \right) \mathbf{V} \in \mathbb{R}^{n \times d_h}.$$

The softmax is row-wise: the i -th output is a weighted average of the value vectors, with weights proportional to the (scaled) inner product between the i -th query and each key. The scaling by $1/\sqrt{d_h}$ counteracts the growth of inner products with dimension, keeping the softmax well-conditioned.

The computational cost is $O(n^2 d_h)$ for the attention matrix and $O(nd^2)$ for the projections. The quadratic dependence on sequence length n is the principal limitation of the transformer; we discuss efficient attention variants below.

10.1.1 Multi-head attention

In practice, attention is computed in parallel by H “heads,” each with its own projections $\mathbf{W}_Q^{(h)}, \mathbf{W}_K^{(h)}, \mathbf{W}_V^{(h)}$ of dimension $d_h = d/H$. The outputs are concatenated and projected back to dimension d :

$$\text{MultiHead}(\mathbf{X}) = \text{Concat}(\text{head}_1, \dots, \text{head}_H) \mathbf{W}_O, \quad \text{head}_h = \text{Attn}(\mathbf{X} \mathbf{W}_Q^{(h)}, \mathbf{X} \mathbf{W}_K^{(h)}, \mathbf{X} \mathbf{W}_V^{(h)})$$

Multi-head attention allows the model to attend to different aspects of the input simultaneously: different heads can specialize to different relations (syntactic, semantic, positional) without explicit supervision. The empirical observation that heads in trained transformers specialize is one of the foundations of mechanistic interpretability (Volume III, Chapter 5).

10.2 Positional encodings

Self-attention is permutation equivariant: it has no notion of order. To incorporate positional information, the input is augmented with **positional encodings**. The original transformer [vaswani2017attention] used sinusoidal encodings of different frequencies:

$$\text{PE}_{i,2k} = \sin(i/10000^{2k/d}), \quad \text{PE}_{i,2k+1} = \cos(i/10000^{2k/d}),$$

where i is the position and k is the dimension. Sinusoidal encodings have the property that $\text{PE}_{i+\delta}$ is a linear function of PE_i , allowing the model to learn to attend by relative position.

Learned absolute positional embeddings (used in BERT, GPT) replace the sinusoidal encoding with a learned vector per position; this is simpler but does not generalize to sequences longer than those seen in training. **Relative positional encodings** [shaw2018self] and **rotary positional encodings (RoPE)** [su2021roformer] encode position through the relative position of queries and keys, and have become the default in modern large language models.

10.3 The transformer block

A transformer block consists of multi-head self-attention followed by a position-wise feedforward network (FFN), each with residual connections and layer normalization:

$$\begin{aligned} \mathbf{X}' &= \text{LayerNorm}(\mathbf{X} + \text{MultiHead}(\mathbf{X})) \\ \mathbf{X}'' &= \text{LayerNorm}(\mathbf{X}' + \text{FFN}(\mathbf{X}')), \end{aligned}$$

where $\text{FFN}(\mathbf{X}) = \text{ReLU}(\mathbf{X}\mathbf{W}_1 + \mathbf{b}_1)\mathbf{W}_2 + \mathbf{b}_2$ is a two-layer MLP applied independently to each position. Modern LLMs (GPT-3, GPT-4) use SwiGLU

or GeGLU activations and pre-norm rather than post-norm.

```
import torch
import torch.nn as nn
import torch.nn.functional as F

class TransformerBlock(nn.Module):
    def __init__(self, d_model, n_heads, d_ff,
        ↪ p_drop=0.1):
        super().__init__()
        self.n_heads = n_heads
        self.d_head = d_model // n_heads
        self.q_proj = nn.Linear(d_model, d_model,
        ↪ bias=False)
        self.k_proj = nn.Linear(d_model, d_model,
        ↪ bias=False)
        self.v_proj = nn.Linear(d_model, d_model,
        ↪ bias=False)
        self.o_proj = nn.Linear(d_model, d_model,
        ↪ bias=False)
        self.ffn = nn.Sequential(
            nn.Linear(d_model, d_ff), nn.GELU(),
            nn.Linear(d_ff, d_model)
        )
        self.norm1 = nn.LayerNorm(d_model)
        self.norm2 = nn.LayerNorm(d_model)
        self.drop = nn.Dropout(p_drop)
    def forward(self, x, mask=None):
        B, N, D = x.shape
        q = self.q_proj(x).view(B, N, self.n_heads,
        ↪ self.d_head).transpose(1, 2)
        k = self.k_proj(x).view(B, N, self.n_heads,
        ↪ self.d_head).transpose(1, 2)
        v = self.v_proj(x).view(B, N, self.n_heads,
        ↪ self.d_head).transpose(1, 2)
        attn = (q @ k.transpose(-1, -2)) /
        ↪ (self.d_head ** 0.5)
        if mask is not None:
            attn = attn.masked_fill(mask == 0,
        ↪ float('-inf'))
        attn = F.softmax(attn, dim=-1)
        out = (attn @ v).transpose(1, 2).reshape(B, N,
        ↪ D)
        x = self.norm1(x +
        ↪ self.drop(self.o_proj(out)))
        x = self.norm2(x + self.drop(self.ffn(x)))
        return x
```

10.4 Encoder, decoder, encoder-decoder

The original transformer [vaswani2017attention] was an **encoder-decoder** architecture for sequence-to-sequence tasks: the encoder processed the source sequence, the decoder generated the target sequence auto-regressively, with **cross-attention** allowing the decoder to attend to the encoder's output.

Three architectural lineages descend from this:

Encoder-only (BERT [devlin2019bert]). Bidirectional attention over the input. Trained with masked language modeling. Best for tasks requiring understanding of the full input (classification, span extraction).

Decoder-only (GPT [radford2018improving]; [brown2020language]). Causal (masked) attention: each position can attend only to earlier positions. Trained with next-token prediction. Best for generation; the architecture of modern large language models.

Encoder-decoder (T5 [raffel2020exploring]; BART [lewis2019bart]). The original architecture, retained for tasks where explicit source-target structure is useful (translation, summarization).

10.5 Efficient attention

The $O(n^2)$ cost of self-attention is the principal computational bottleneck of transformers, especially for long sequences. Three families of approximations have been developed.

Sparse attention. Restrict attention to a subset of position pairs (local window, strided, or learned). [child2019generating] showed that $O(n\sqrt{n})$ sparse attention is sufficient for image generation.

Linear attention. Replace the softmax with a kernel feature map ϕ such that $\text{Attn} = (\phi(\mathbf{Q})\phi(\mathbf{K})^\top)\mathbf{V} = \phi(\mathbf{Q})(\phi(\mathbf{K})^\top\mathbf{V})$, reducing the cost from $O(n^2d_h)$ to $O(nd_h^2)$. [katharopoulos2020linear] is a representative example.

Low-rank attention. Approximate the attention matrix by a low-rank factorization. [wang2020linformer] projects the key and value matrices to a fixed dimension.

A different line of work avoids approximating attention and instead computes it exactly with $O(n)$ memory via **flash attention** [dao2022flashattention], which exploits the structure of GPU memory hierarchies to compute exact attention with minimal memory traffic.

10.6 Computational experiments

10.6.1 Experiment 1: Attention patterns

Train a small transformer on a simple algorithmic task (e.g., sorting a sequence of numbers). Visualize the attention weights of each head. Are the patterns interpretable?

10.6.2 Experiment 2: Positional encoding comparison

Train a small transformer with sinusoidal, learned absolute, and rotary positional encodings on a sequence classification task. Compare generalization to longer sequences than seen in training.

10.6.3 Experiment 3: Scaling

Train transformers of varying size (depth, width, parameter count) on a language modeling task. Plot training loss as a function of parameter count. Verify the scaling laws of Chapter 9 (Volume III).

10.7 Historical notes

Self-attention was introduced to neural machine translation by [bahdanau2015neural] as an additive attention mechanism over encoder hidden states. The fully attention-based transformer is due to [vaswani2017attention], who showed that attention alone (without recurrence) is sufficient for state-of-the-art translation. BERT [devlin2019bert] demonstrated the power of bidirectional pretraining. GPT-2 [radford2019language] and GPT-3 [brown2020language] demonstrated the scaling behavior of decoder-only language models. The scaling laws of [kaplan2020scaling] and [hoffmann2022training] (Volume III, Chapter 4) provided the empirical foundation for the subsequent scale-up. Flash attention [dao2022flashattention] made exact long-sequence attention practical.

10.8 Exercises

1. **(Easy)** Show that self-attention is permutation equivariant. Why does this necessitate positional encodings?
2. **(Easy)** Compute the parameter count and FLOPs of a transformer block with $d_{\text{model}} = 512$, $H = 8$ heads, $d_{\text{ff}} = 2048$, sequence length $n = 1024$.
3. **(Medium)** Implement sinusoidal, learned, and rotary positional encodings. Train a small transformer with each on a sequence classification task. Compare.
4. **(Medium)** Implement multi-head attention without using `nn.MultiheadAttention`. Verify that

your implementation produces the same output as PyTorch's.

5. **(Medium)** Implement masked (causal) attention for autoregressive language modeling. Train a small GPT-like model on a text dataset.
6. **(Hard)** Implement linear attention. Compare its accuracy and runtime to standard softmax attention on a long-sequence task.
7. **(Hard)** Implement flash attention. Verify that it produces the same output as standard attention, with lower memory usage.
8. **(Research)** Investigate the attention patterns of a pretrained model (e.g., GPT-2 small) on natural text. Are specific heads specialized to specific syntactic or semantic phenomena?

Generative Models

A generative model is one that learns to produce samples from a data distribution p_{data} . The three principal families — variational autoencoders (VAEs), generative adversarial networks (GANs), and diffusion models — represent three different solutions to the same fundamental problem: how to learn a probability distribution from samples, when the distribution is supported on a high-dimensional space (e.g., $\mathbb{R}^{3 \times 256 \times 256}$ for images) and is accessible only through samples.

The three families differ in what they explicitly represent. VAEs learn an explicit **encoder** and **decoder** that map between data and a latent space; the density model is implicit but the latent representation is explicit. GANs learn a **generator** that maps noise to data; the density model is implicit and no encoder is learned. Diffusion models learn the **reverse of a noise-adding process**; the density model is explicit (up to a normalizing constant) and a score function is learned.

11.1 Variational autoencoders

The VAE [kingma2013auto]; [rezende2014stochastic] combines an **encoder** $q_\phi(\mathbf{z}|\mathbf{x})$ mapping data to a latent distribution, a **decoder** $p_\theta(\mathbf{x}|\mathbf{z})$ mapping latents to data, and a **prior** $p(\mathbf{z})$ over the latent space (typically $\mathcal{N}(\mathbf{0}, \mathbf{I})$). Training maximizes the **evidence lower bound** (ELBO; see Chapter 4):

$$\mathcal{L}_{\text{VAE}}(\theta, \phi) = \mathbb{E}_{q_\phi(\mathbf{z}|\mathbf{x})}[\log p_\theta(\mathbf{x}|\mathbf{z})] - D_{\text{KL}}(q_\phi(\mathbf{z}|\mathbf{x}) \| p(\mathbf{z})).$$

The first term rewards the decoder for reconstructing the data from the latent; the second term regularizes the encoder’s posterior toward the prior. The reparameterization trick makes the objective differentiable with respect to both θ and ϕ .

VAEs are attractive for the explicit, structured latent space they learn: latents can be interpolated, disentangled, and conditioned on. Their principal drawback is **posterior collapse**: the encoder may ignore the input and match the prior, in which case the decoder learns a marginal model and samples lack diversity. Various mitigations (KL annealing, free bits, stronger decoders) have been proposed.

11.2 Generative adversarial networks

The GAN [NIPS2014_5dce8a06] formulates generation as a two-player game between a **generator** G_θ (mapping noise $\mathbf{z} \sim p(\mathbf{z})$ to data) and a **discriminator** D_ψ (classifying real vs. fake). The generator tries to fool the discriminator; the discriminator tries to detect fakes. The minimax objective is

$$\min_{\theta} \max_{\psi} \mathbb{E}_{\mathbf{x} \sim p_{\text{data}}} [\log D_\psi(\mathbf{x})] + \mathbb{E}_{\mathbf{z} \sim p(\mathbf{z})} [\log(1 - D_\psi(G_\theta(\mathbf{z})))].$$

At the Nash equilibrium, the generator produces samples indistinguishable from the data and the discriminator outputs $1/2$ everywhere. In practice, GANs produce sharper samples than VAEs but are notoriously hard to train: the minimax game is unstable, and the equilibrium is rarely reached cleanly. Mode collapse (the generator produces only a subset of the data modes) is the principal failure mode.

The empirical state of GAN training was substantially improved by **Wasserstein GAN** [arjovsky2017wasserstein], which replaces the Jensen-Shannon divergence with the Wasserstein-1 distance, and by **style-based architectures** [karras2019style], which substantially improved sample quality.

11.3 Diffusion models

A diffusion model [sohl2015deep]; [ho2020denoising] defines a **forward process** that gradually adds Gaussian noise to the data over T steps:

$$q(\mathbf{x}_t | \mathbf{x}_{t-1}) = \mathcal{N}(\mathbf{x}_t; \sqrt{1 - \beta_t} \mathbf{x}_{t-1}, \beta_t \mathbf{I}),$$

where $\beta_1, \dots, \beta_T$ is a variance schedule. The forward process has a closed form:

$$q(\mathbf{x}_t | \mathbf{x}_0) = \mathcal{N}(\mathbf{x}_t; \sqrt{\bar{\alpha}_t} \mathbf{x}_0, (1 - \bar{\alpha}_t) \mathbf{I}), \quad \bar{\alpha}_t = \prod_{s=1}^t (1 - \beta_s).$$

A neural network $\epsilon_\theta(\mathbf{x}_t, t)$ is trained to predict the noise added at step t , by minimizing

$$\mathcal{L}_{\text{diffusion}}(\theta) = \mathbb{E}_{t, \mathbf{x}_0, \epsilon} [\|\epsilon - \epsilon_\theta(\sqrt{\bar{\alpha}_t} \mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon, t)\|^2],$$

where $\epsilon \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$. Sampling reverses the forward process via a Markov chain that starts from $\mathbf{x}_T \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ and iteratively denoises:

$$\mathbf{x}_{t-1} = \frac{1}{\sqrt{1-\beta_t}} \left(\mathbf{x}_t - \frac{\beta_t}{\sqrt{1-\bar{\alpha}_t}} \epsilon_{\theta}(\mathbf{x}_t, t) \right) + \sigma_t \mathbf{z},$$

with $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$. The score-based perspective [song2021scorebased] unifies diffusion models with score matching, showing that the network learns the score $\nabla_{\mathbf{x}} \log p_t(\mathbf{x})$ of the noise-perturbed data distribution.

Diffusion models have become the dominant paradigm for image generation since 2020, owing to their stable training (a simple squared-error objective) and high sample quality. Their principal drawback is the slow sampling: $T \sim 1000$ denoising steps are required for high-quality samples, though distillation and fast samplers reduce this to 4–10 steps with modest quality loss.

```
import torch

[torch].no_grad()
def ddpm_sample(model, shape, betas, alphas_cumprod,
    ↪ device='cpu'):
    """Reverse diffusion process: sample  $x_0$  from
    ↪ noise  $x_T$ ."""
    T = len(betas)
    x = torch.randn(shape, device=device)
    for t in reversed(range(T)):
        beta_t = betas[t]
        alpha_bar_t = alphas_cumprod[t]
        eps_pred = model(x, torch.full((shape[0],), t,
    ↪ device=device))
        mean = (1 / (1 - beta_t).sqrt()) * (x -
    ↪ (beta_t / (1 - alpha_bar_t).sqrt()) * eps_pred)
        if t > 0:
            noise = torch.randn_like(x)
            x = mean + betas[t].sqrt() * noise
        else:
            x = mean
    return x
```

11.4 Computational experiments

11.4.1 Experiment 1: VAE on MNIST

Train a VAE on MNIST. Plot reconstructions and samples. Vary the KL weight and observe the reconstruction-vs-regularization trade-off.

11.4.2 Experiment 2: GAN on a small dataset

Train a DCGAN on a small image dataset (e.g., CIFAR-10). Plot samples and discriminator/generator loss curves. Observe mode collapse if it occurs.

11.4.3 Experiment 3: Diffusion on a 2D toy

Train a small diffusion model on a 2D toy distribution (e.g., a mixture of Gaussians). Visualize the forward process and the reverse sampling trajectory.

11.5 Historical notes

The VAE was introduced by [kingma2013auto] and [rezende2014stochastic] in 2013–2014. The GAN is due to [NIPS2014_5dce8a06]. Diffusion models were introduced by [sohl2015deep] and brought to maturity by [ho2020denoising]. The score-based unification is due to [song2021scorebased]. The combination of diffusion models with classifier-free guidance [ho2022classifier] launched the era of high-quality text-to-image generation.

11.6 Exercises

1. **(Easy)** Derive the VAE objective (ELBO) from the marginal log-likelihood.
2. **(Easy)** Show that the GAN objective is the Jensen-Shannon divergence at the optimal discriminator.
3. **(Medium)** Implement a VAE on MNIST. Investigate posterior collapse: under what conditions does the encoder ignore the input?
4. **(Medium)** Implement a DCGAN. Train on CIFAR-10. Investigate the effect of the discriminator-to-generator update ratio.
5. **(Medium)** Implement a small DDPM. Train on MNIST. Investigate the effect of the number of timesteps T .
6. **(Hard)** Implement Wasserstein GAN with gradient penalty. Compare to vanilla GAN in terms of training stability and sample quality.
7. **(Hard)** Implement classifier-free guidance for a class-conditional diffusion model. Quantify the effect of the guidance scale on sample quality.
8. **(Research)** Investigate the score-based perspective on diffusion models. What does the learned score function look like for a simple 1D toy distribution?

The Practice of Training

This chapter treats the engineering of large-scale training: the design decisions that, while less theoretically clean than the material of preceding chapters, are essential for actually training modern deep networks. The chapter is organized around the training loop: initialization, optimization, normalization, regularization, hyperparameter selection, distributed training, and the monitoring of training runs. Each section states the canonical practice, the empirical justification, and the failure modes.

12.1 Initialization revisited

The initialization schemes of Chapter 5 (Xavier, He) were derived for MLPs under simplifying assumptions. Modern architectures require modifications:

- **Transformer initialization** typically uses Xavier with a small additional scaling factor on the deeper layers (e.g., $\sqrt{2/L}$ scaling), or initializes the output layer of each residual block to zero, so that the network is initially the identity. The latter was introduced by [gpt2paper] and is now standard.
- **Embedding initialization** is typically $\mathcal{N}(0, 1/\sqrt{d_{\text{model}}})$, with optional truncation.
- **Output projection** is often initialized to zero or to a small constant, to ensure the initial loss matches the uniform-distribution baseline.

12.2 Optimizer selection

AdamW [loshchilov2019decoupled] — Adam with decoupled weight decay — is the dominant optimizer for transformer training. The decoupling separates the gradient update from the weight decay, which empirically improves generalization. The canonical hyperparameters are $\beta_1 = 0.9$, $\beta_2 = 0.95\text{--}0.999$ (lower for large models), $\varepsilon = 10^{-8}$, weight decay 0.1, and a learning rate that is warmup-then-cosine-decay.

The **learning rate schedule** is consequential. The standard schedule is a linear warmup over N_{warmup} steps (typically 0.1–1% of total steps), followed by a cosine decay to 10–0% of the peak learning rate. The peak learning rate is set by scaling laws: for a model of size N trained on a dataset of size D with batch size B , the optimal peak learning rate scales roughly as $\min(N, D)^{-0.5}$.

```
import torch

def get_lr(step, warmup, n_total, lr_max):
    if step < warmup:
        return lr_max * step / warmup
    progress = (step - warmup) / max(1, n_total - warmup)
    return 0.1 * lr_max + 0.45 * lr_max * (1 +
        math.cos(math.pi * progress))
```

12.3 Normalization revisited

Layer normalization (Chapter 5) is the default in transformers. The choice of **pre-norm vs. post-norm** — whether the normalization is applied before or after the residual branch — has implications for stability. Post-norm (the original transformer) is harder to train at scale; pre-norm is now standard. **RMSNorm** [zhang2019rootmean], which drops the mean-subtraction and only rescales by the RMS, is a common simplification used in LLaMA and other modern transformers.

12.4 Regularization

In addition to weight decay, modern training uses several regularizers:

- **Dropout** remains useful in non-attention layers, though it has been displaced by data augmentation and scale in many transformer setups.
- **Label smoothing** [szegedy2016rethinking] replaces the one-hot target distribution with a smoothed distribution ($y_i = (1 - \alpha) + \alpha/K$ for the correct class, $y_i = \alpha/K$ for others). It acts as a calibrator and is universal in modern training.
- **Stochastic depth** [huang2016deep] randomly drops entire residual blocks during training, providing both regularization and computational savings.
- **Data augmentation** is the single most impactful regularizer for vision. For text, augmentation is harder; paraphrasing and back-translation are common.

12.5 Distributed training

Training at the scale of modern large language models (LLaMA-3 405B, GPT-4) requires thousands of GPUs. The principal strategies are:

Data parallelism (DP). Each GPU holds a complete copy of the model; each processes a differ-

ent mini-batch; gradients are averaged across GPUs. Limited by the requirement that the model fits in a single GPU's memory.

Tensor parallelism (TP). The model's weight matrices are split across GPUs; each GPU computes a slice of each matrix multiplication; intermediate results are synchronized by all-reduce. Limited by communication overhead.

Pipeline parallelism (PP). The model is split layer-wise across GPUs; activations are passed from one GPU to the next as in a pipeline. Limited by "bubbles" where GPUs wait for the next batch to arrive.

Fully sharded data parallelism (FSDP) [zero]. Each GPU holds a shard of the model's parameters, gradients, and optimizer states; before each forward/backward pass, the relevant shard is all-gathered, used, and discarded. Combines the memory efficiency of model parallelism with the simplicity of data parallelism.

In practice, modern large-scale training uses a 3D combination: data parallelism across nodes, tensor parallelism within nodes, and pipeline parallelism across model depth. The exact recipe is a function of the model size, the cluster topology, and the desired throughput.

12.6 Monitoring training

The principal indicators of healthy training are:

- **Loss curve:** smooth decay, no divergence, no plateau (unless the model has converged).
- **Gradient norm:** stable; sudden spikes indicate instability.
- **Activation statistics:** mean near 0, variance near 1 after each normalization layer.
- **Attention entropies:** distribution of attention weights across heads; should not collapse.
- **Validation loss:** should track training loss; divergence indicates overfitting.
- **Learning rate:** actual vs. scheduled, to verify the schedule is correctly applied.

The most common pathologies are: loss divergence (learning rate too high, gradient clipping too lax), loss plateau (learning rate too low, optimizer stuck), gradient explosion (gradient clipping too lax, numerical instability), and overfitting (training

loss continues to decrease while validation loss increases). Each has standard fixes that should be applied systematically.

12.7 Computational experiments

12.7.1 Experiment 1: Learning rate sweep

Train a small transformer on a language modeling task with a sweep of peak learning rates (10^{-4} , 10^{-3} , 10^{-2}). Plot training loss curves. Identify the stable range.

12.7.2 Experiment 2: Weight decay ablation

Train the same model with weight decay $\in \{0, 0.01, 0.1, 1.0\}$. Plot training and validation loss. Identify the regularization effect.

12.7.3 Experiment 3: Distributed training

Use PyTorch's `DistributedDataParallel` to train a small model on multiple GPUs (or simulate on a single GPU). Measure the throughput scaling as a function of GPU count.

12.8 Historical notes

The canonical practice of training large transformers was developed in the GPT [radford2018improving], [radford2019language], [brown2020language] line of work, refined by LLaMA [touvron2023llama] and successors. AdamW is due to [loshchilov2019decoupled]. RMSNorm to [zhang2019rootmean]. Stochastic depth to [huang2016deep]. The FSDP strategy is due to [zero]. The 3D parallelism recipe is documented in the training infrastructure papers of Megatron-LM [shoeybi2019megatron] and GPT-3 [brown2020language]. Flash attention [dao2022flashattention] is essential for efficient long-context training.

12.9 Exercises

1. **(Easy)** Derive the cosine learning rate schedule. What is the gradient at the start and end of the schedule?

2. **(Easy)** Show that label smoothing with α corresponds to a KL regularization toward the uniform distribution with weight α .
3. **(Medium)** Implement AdamW from scratch. Verify that it matches PyTorch's implementation on a synthetic problem.
4. **(Medium)** Implement RMSNorm. Compare its training dynamics to LayerNorm on a small transformer.
5. **(Medium)** Implement stochastic depth. Train a ResNet with and without it; compare test accuracy.
6. **(Hard)** Implement data parallelism from scratch (using `torch.distributed`). Verify that gradients are correctly averaged.
7. **(Hard)** Implement tensor parallelism for a transformer block. Measure the communication overhead as a function of model size.
8. **(Research)** Investigate the training stability of large models. What goes wrong, and how can it be mitigated? (See [mosbach2021stability], [chowdhery2022palm] for starting points.)

Part III.

Frontiers and Open Problems

Introduction to Volume III

Volume III turns from the well-established core of deep learning to the open frontier. Each chapter takes a question that the field has not yet settled — what does a deep network’s representation *mean*? why do larger models *scale*? when does an overparameterized network *generalize*? can a trained network be *understood mechanistically*? — and surveys the current state of the art, with attention to both what is known and what is contested. The volume closes with a chapter of explicit open problems, twenty of them, that we believe will shape the next decade of research.

The unifying perspective is that **deep learning is, in 2025, an empirical science in search of its theory**. The empirical record is rich and growing; the theory is partial and contested. This volume treats the principal candidates for that theory with intellectual honesty: where the theory is illuminating, we say so; where it is contested or vacuous, we say that too.

13.1 Chapter map

Chapter 11 — Representation Learning. The geometry of the feature spaces learned by deep networks. Manifold hypotheses, linear representations, superposition, polysemantic neurons, the role of sparse coding.

Chapter 12 — Scaling Laws. The empirical regularity that loss decreases as a power law in compute, model size, and data; the theoretical explanations that have been proposed; the implications for compute-optimal training.

Chapter 13 — Generalization Theory. Why do overparameterized networks generalize? The classical theory (uniform convergence, Rademacher complexity) is vacuous; the modern candidates (implicit bias, PAC-Bayes, compression, NTK) are partial. We treat each with its strengths and limitations.

Chapter 14 — Interpretability. The program of reading networks from the inside. Circuits, superposition, sparse autoencoders, mechanistic interpretability, the “microscope” vs. “MRI” distinction.

Chapter 15 — Open Problems. Twenty problems, organized by area, with the current state of knowledge and the principal obstacles.

13.2 How to read

The chapters of this volume are largely independent. Chapter 11 assumes Volume II; Chapter 12 assumes Volume I (optimization) and Volume II (architectures); Chapter 13 assumes Volume I (probability) and Volume II; Chapter 14 assumes Volume II and Chapter 11. Chapter 15 assumes everything.

Representation Learning

What does a deep network *know*? The question has at least three readings. The first asks what *function* the network computes — answered by querying it on inputs. The second asks what *features* the network has learned — answered by inspecting intermediate activations. The third asks what *concepts* those features correspond to — answered, when it can be answered at all, by careful probing and causal intervention. This chapter treats the second and third readings, surveying the empirical and theoretical understanding of the representations learned by deep networks.

14.1 The manifold hypothesis

Natural data — images, audio, text — are widely believed to lie on or near a low-dimensional manifold embedded in the high-dimensional ambient space. The empirical evidence is twofold: (i) local dimensionality estimators applied to natural data give dimensions much smaller than the ambient (e.g., 30–100 for natural image patches, vs. an ambient dimension of 768 for 16×16 RGB patches); (ii) generative models with low-dimensional latent spaces (VAEs, GANs, diffusion models) generate convincing samples, suggesting that the data distribution is well-approximated by a low-dimensional projection.

Definition (Manifold hypothesis)

The support of the data distribution p_{data} on $\mathbb{R}^D$ is contained, up to a set of measure zero, in a d -dimensional smooth submanifold $\mathcal{M} \subset \mathbb{R}^D$ with $d \ll D$.

The manifold hypothesis is an empirical claim, not a theorem. It is contested: [pope2021intrinsic] measured intrinsic dimensions across many datasets and found substantial variation; some data modalities (text in particular) may not satisfy the hypothesis in any simple form. Nonetheless, the hypothesis is the right organizing framework for thinking about representations: a deep network’s job is, on this view, to *identify the manifold* — to map from $\mathbb{R}^D$ to a coordinate system on $\mathcal{M}$.

14.2 Linear representations

A striking empirical regularity, documented most clearly in modern large language models, is that **semantic concepts are often linearly encoded**

in the activations. The classic example is gender in word embeddings ([mikolov2013distributed]): $\text{vec}(\text{king}) - \text{vec}(\text{man}) \approx \text{vec}(\text{queen}) - \text{vec}(\text{woman})$, with a similar arithmetic for capital-country relations, verb tense, and so on. More recently, the same linear structure has been found in transformer activations for syntactic categories, factual knowledge, and behavioral properties.

The **linear representation hypothesis** [park2024linear] codifies this: concepts correspond to directions in activation space, and concept *strength* corresponds to projection along the direction. The hypothesis has substantial empirical support and is the foundation of much current mechanistic interpretability work.

14.3 Superposition and polysemantic neurons

A central puzzle is the **polysemanticity** of neurons: individual units in trained networks often respond to multiple, apparently unrelated concepts. A single neuron in a vision model might fire for cat faces, car fronts, and the letter “S” — a striking lack of the clean one-neuron-one-concept mapping that early interpretability work hoped for.

[elhage2022superposition] proposed the **superposition hypothesis**: networks represent more features than they have neurons by storing features in *superposition*, with features encoded as nearly-orthogonal directions in a high-dimensional activation space. The geometry is approximately that of the Johnson–Lindenstrauss lemma (Chapter 1): N features can be stored in $d \ll N$ dimensions with low interference, provided the features are *sparse* (only a small fraction are active at any given input).

The superposition hypothesis has substantial explanatory power: it explains polysemanticity, the empirical success of sparse coding, and the emergence of clean monosemantic features under sparse autoencoders. It also poses a clean theoretical question — when does a network prefer sparse superposition over dense one-to-one representation? — that is partially answered by [anthropic2024scaling].

14.4 Sparse autoencoders

A **sparse autoencoder** (SAE) is a tool for recovering the underlying features from a network’s activations. Given activations $\mathbf{x} \in \mathbb{R}^d$, an SAE computes

$$\mathbf{z} = \text{ReLU}(\mathbf{W}_{\text{enc}}\mathbf{x} + \mathbf{b}), \quad \hat{\mathbf{x}} = \mathbf{W}_{\text{dec}}\mathbf{z} + \mathbf{b}',$$

with $|\mathbf{z}|_0$ small (sparsity penalty) and the dictionary $\mathbf{W}_{\text{dec}} \in \mathbb{R}^{d \times N}$ having $N \gg d$ columns. The columns of $\mathbf{W}_{\text{dec}}$ are the learned **features**, and the sparse code $\mathbf{z}$ tells which features are present in $\mathbf{x}$.

Empirically, SAEs trained on the activations of large language models recover a substantial number of **monosemantic** features — features that respond to a single, identifiable concept. The recovered features include syntactic roles (subject, object, verb), semantic categories (country, code, math), and behavioral properties (the network’s intent to refuse, to flatter, to confabulate). The state of the art is rapidly evolving; see [templeton2024scaling] for a recent large-scale study.

14.5 Probing and causal interventions

A **linear probe** is a simple classifier trained on a network’s activations to predict some property of the input. A high probing accuracy indicates that the property is linearly decodable from the activations — that the network “knows” about the property in some operational sense. Probing is widely used but has a key limitation: it tests whether the property is *represented*, not whether the network *uses* the representation. A network might represent a property it does not use, or use a property it does not linearly represent.

Causal interventions address this limitation. The simplest is **activation patching**: replace the activations of a network at a specific layer and position with those from a different input, and observe the effect on the output. If the patch changes the output in the expected direction, the patched activations causally mediate the relevant computation. **Ablation** is the special case of patching with zeros.

A more refined intervention is **path patching**, which patches only the activations flowing along specific paths through the network. Path patching

has been used to identify the circuits by which transformers implement specific syntactic computations, and is a principal tool of mechanistic interpretability (Chapter 14).

14.6 Computational experiments

14.6.1 Experiment 1: Word embedding arithmetic

Load a pretrained word embedding (word2vec, GloVe). Verify the king–queen analogy and find three more analogies that work quantitatively.

14.6.2 Experiment 2: Polysemanticity

Train a small CNN on a multi-task image dataset (e.g., CIFAR-10 + a synthetic attribute). Visualize the top activations of neurons in the penultimate layer; observe that many neurons respond to multiple concepts.

14.6.3 Experiment 3: Sparse autoencoder

Train a small SAE on the activations of a pretrained model (e.g., ResNet18 on ImageNet). Inspect the learned features; quantify monosemanticity by correlation with known concepts.

14.7 Historical notes

The manifold hypothesis has roots in the early vision literature; [tenenbaum2000global] and [roweis2000nonlinear] introduced the modern dimensionality-reduction framework. Linear representation in word embeddings was emphasized by [mikolov2013distributed]. The superposition hypothesis is due to [elhage2022superposition]; the sparse-autoencoder methodology was developed by [bricken2023monosemanticity] and [templeton2024scaling]. Activation patching as a causal intervention was introduced by [geiger2021causal] and [vig2020investigating]; path patching by [wang2022interpretability].

14.8 Exercises

1. (Easy) Implement a linear probe. Train it on the activations of a small CNN to predict the class label. What does the probing accuracy tell you? What does it not tell you?

2. **(Easy)** Implement activation patching. Verify that patching the activations of a transformer at a specific position changes the output in the expected direction.
3. **(Medium)** Train a sparse autoencoder on a small model's activations. Quantify the trade-off between reconstruction error and sparsity.
4. **(Medium)** Estimate the intrinsic dimensionality of a small image dataset (e.g., CIFAR-10) using a nearest-neighbor-based estimator.
5. **(Medium)** Implement a linear concept erasure operation: given a direction in activation space, project it out and observe the effect on the network's behavior.
6. **(Hard)** Reproduce a superposition experiment: train a small ReLU autoencoder on a synthetic sparse dataset with more features than dimensions. Verify that the learned dictionary approximates the original features.
7. **(Hard)** Implement path patching. Use it to identify the circuit by which a small transformer solves a simple algorithmic task (e.g., induction).
8. **(Research)** Investigate the relationship between superposition and feature geometry. How does the angle between learned features relate to their co-occurrence statistics?

Scaling Laws

One of the most striking empirical regularities of modern deep learning is that **training loss decreases as a power law in compute, model size, and data**. This regularity — a *scaling law* — has reshaped the field since 2020, transforming model development from an artisanal process of architecture search into an engineering discipline of scale-up. This chapter surveys the empirical laws, the theoretical explanations that have been proposed, and the implications for compute-optimal training.

15.1 The Kaplan scaling law

[kaplan2020scaling] measured the test loss of decoder-only transformers trained on a large text corpus, varying model size N (parameters), dataset size D (tokens), and compute C (FLOPs). They found that, when not bottlenecked by the other factors, the loss scales as a power law:

$$L(N) = \left(\frac{N_c}{N}\right)^{\alpha_N}, \quad L(D) = \left(\frac{D_c}{D}\right)^{\alpha_D}, \quad L(C) = \left(\frac{C_c}{C}\right)^{\alpha_C}$$

with exponents $\alpha_N \approx 0.076$, $\alpha_D \approx 0.095$, $\alpha_C \approx 0.05$. The scaling is smooth over six orders of magnitude in N and D , with no detectable departures — a remarkable regularity.

The Kaplan law implies that the path to better models is straightforward: scale up. It also implies a specific *compute-optimal* allocation: given a fixed compute budget C , how should one trade off N and D ? Kaplan et al. concluded that model size should grow faster than data — that very large models trained on relatively modest data were compute-optimal. This conclusion drove the development of models like GPT-3 [brown2020language] (175B parameters trained on 300B tokens).

15.2 The Chinchilla correction

[hoffmann2022training] revisited the Kaplan scaling law with a much larger set of training runs and a refined methodology. Their headline finding was that **Kaplan et al. under-allocated data**: compute-optimal training requires data to scale roughly linearly with model size, not sub-linearly. The corrected scaling law is

$$L(N, D) = E + \frac{A}{N^\alpha} + \frac{B}{D^\beta},$$

with $\alpha \approx 0.34$, $\beta \approx 0.28$, E an irreducible entropy term, and A, B constants. The compute-optimal allocation is approximately $N \propto C^{0.5}$, $D \propto C^{0.5}$: model size and data should scale in equal proportion.

The practical implication is dramatic: a compute-optimal Chinchilla model (70B parameters, 1.4T tokens) matches or exceeds the performance of GPT-3 (175B parameters, 300B tokens) while using $4\times$ less compute. The Kaplan-GPT-3 allocation was, in retrospect, a substantial over-investment in parameters at the expense of data. Subsequent large models (LLaMA, PaLM, Mistral) have largely adopted the Chinchilla allocation.

15.3 Scaling beyond Chinchilla

Several extensions and refinements of the Chinchilla law have been studied:

Scaling of downstream task performance. The Kaplan and Chinchilla laws describe *next-token prediction loss*, not task performance. The mapping from loss to task accuracy is approximately sigmoidal: a given loss reduction corresponds to a larger accuracy gain when the model is near the “threshold” of capability and a smaller gain when the model is far from the threshold. The emergence of new capabilities at scale — a phenomenon documented by [wei2022emergent] — is partly an artifact of this sigmoid mapping and partly a real change in the model’s behavior.

Data-constrained scaling. The Chinchilla law assumes unlimited data. In practice, high-quality text data is finite and may be exhausted at scales of 10^{13} – 10^{14} tokens. [muennighoff2023scaling] extended the scaling law to data-constrained regimes, where the law breaks down and additional data must be synthesized, translated, or replayed.

Compute-optimal inference. At inference time, the cost of generating each token depends on the model size. For a fixed total inference budget, smaller models running longer can outperform larger models running shorter; the optimal model size for inference is much smaller than for training. This is the foundation of the **distillation** paradigm: train a large model, distill to a small one, deploy the small one.

15.4 Theoretical explanations

Several theoretical explanations for the empirical scaling laws have been proposed, none fully satisfactory.

Statistical mechanics. [bahri2021statistical] and others have derived scaling laws from a statistical-mechanics analysis of the loss landscape, treating the network as a disordered system and the loss as a free energy. The exponents fall out of the analysis under certain assumptions; the assumptions are restrictive.

Information-theoretic. The irreducible loss E is naturally interpreted as the entropy of the data; the reducible loss $(A/N^\alpha + B/D^\beta)$ is the *complexity* of approximating the data distribution with a network of size N trained on D samples. The exponents α, β are harder to derive from information-theoretic arguments.

Random features and kernel methods. In the infinite-width limit, neural networks become kernel methods (the NTK, Chapter 13), and the scaling laws of kernel methods can be computed explicitly. The result matches the empirical exponents qualitatively but not quantitatively.

The honest state of the field is that **scaling laws are an empirical regularity in search of a theory**. The laws themselves are robust — they have been reproduced across many labs, datasets, and architectures — but the underlying mechanism is not understood.

15.5 Implications for compute-optimal training

The practical implications of scaling laws are:

1. **Scale predictably.** Given a compute budget, the Chinchilla law predicts the achievable loss and the optimal model/data split. This makes large-scale training a planning exercise rather than a research gamble.
2. **Allocate data generously.** The Chinchilla correction showed that data should scale with model size; the field has largely corrected the under-allocation of the GPT-3 era.
3. **Watch for breakdowns.** The laws assume unlimited high-quality data; as data becomes con-

strained, the laws break down and the marginal return on compute falls.

4. **Plan for inference.** The training compute-optimal model is not the inference compute-optimal model; distillation and quantization are essential parts of the deployment pipeline.

15.6 Computational experiments

15.6.1 Experiment 1: Kaplan scaling on a small dataset

Train transformers of varying size (e.g., 10M, 30M, 100M, 300M parameters) on a fixed text dataset. Plot the final training loss as a function of parameter count on a log-log scale. Verify the power-law scaling.

15.6.2 Experiment 2: Compute-optimal allocation

For a fixed compute budget, train models of varying sizes with appropriate data. Identify the compute-optimal allocation. Compare to the Chinchilla prediction.

15.6.3 Experiment 3: Emergence

Train a model on a task with a clear threshold (e.g., modular arithmetic). Plot accuracy as a function of compute. Verify the sigmoid-like emergence.

15.7 Historical notes

The empirical study of scaling in deep learning predates the transformer: [hestness2017deep] reported power-law scaling of test loss with data for a range of models. The transformer-specific scaling law is due to [kaplan2020scaling]. The Chinchilla correction is due to [hoffmann2022training]. The emergence phenomenon was systematically documented by [wei2022emergent]. The statistical-mechanics perspective is developed in [bahri2021statistical]. The data-constrained regime is treated in [muenighoff2023scaling].

15.8 Exercises

1. (Easy) State the Chinchilla scaling law. What are the values of the exponents α, β ?

2. **(Easy)** Compute the compute-optimal allocation of model and data for a budget of 10^{22} FLOPs, using the Chinchilla exponents.
3. **(Medium)** Reproduce a small-scale scaling experiment: train models of 4 sizes on a fixed dataset, fit the Kaplan and Chinchilla laws, compare the fits.
4. **(Medium)** Investigate emergence: train a model on a synthetic task with a clear capability threshold (e.g., n -digit addition). Plot accuracy as a function of model size and identify the emergence scale.
5. **(Medium)** Implement a data replay strategy: train a model on a fixed dataset for multiple epochs. Does the scaling law continue to hold, or does the law break down?
6. **(Hard)** Derive the compute-optimal allocation from the Chinchilla law. Show that $N \propto C^a$, $D \propto C^b$ with $a + b = 1$, and find a, b .
7. **(Hard)** Investigate the breakdown of scaling laws in the data-constrained regime. Train a model on a small dataset for many epochs. How does the loss scale with compute?
8. **(Research)** Investigate the scaling of *downstream* task performance (rather than next-token loss). How does it differ from the scaling of the loss? Are the exponents the same?

Generalization Theory

The central theoretical puzzle of deep learning is generalization: why do networks trained to interpolate their training data — networks with sufficient capacity to memorize random labels [zhang2021understanding] — generalize to unseen test data? Classical learning theory, based on uniform convergence and capacity measures (VC dimension, Rademacher complexity), is *vacuous* for overparameterized networks: it predicts a generalization gap of order $O(\sqrt{P/N})$, which is $\gg 1$ for modern networks. This chapter surveys the candidate explanations that have emerged since 2017, treating each with its strengths and limitations.

16.1 The classical theory is vacuous

As reviewed in Chapter 3, classical uniform convergence gives bounds of the form

$$\mathcal{L}(\theta) \leq \hat{\mathcal{L}}(\theta) + O\left(\sqrt{\frac{P \log N}{N}}\right),$$

where P is the parameter count and N is the dataset size. For a modern network with $P = 10^9$ parameters trained on $N = 10^9$ examples, this bound is $\gg 1$ — useless.

Worse, [belkin2019reconciling] and [belkin2018understanding] showed that the classical bias-variance tradeoff (small models underfit, large models overfit) breaks down in the overparameterized regime: test error *decreases* beyond the interpolation threshold. This **double descent** phenomenon is inconsistent with the classical theory, which predicts monotone increase of test error with capacity in the overfitting regime.

16.2 Implicit bias of SGD

The first modern candidate explanation is **implicit bias**: among all the parameter settings that fit the training data, SGD preferentially finds solutions with low complexity. The complexity measure depends on the architecture and optimizer:

- For **linear regression** in the overparameterized regime, gradient descent on the squared loss converges to the **minimum-norm** interpolating solution [zhang2021understanding]. This is the

maximum-margin solution under appropriate conditions, and it generalizes well.

- For **linear classifiers** trained with logistic loss on separable data, gradient descent converges to the **maximum-margin** (L2-SVM) solution [soudry2018implicit].
- For **deep networks**, the situation is more complex: the implicit bias depends on the architecture, initialization, optimizer, and training duration. For wide networks, the bias is toward the **NTK regime** solution (Chapter 4); for narrower networks, the bias is toward flatter minima, sparser activations, or lower-rank weight matrices, depending on the setup.

The implicit bias program has been the most productive vein of modern generalization theory. It is partial — the bias is well-characterized only in restricted regimes — but it provides a clean conceptual framework.

16.3 Flat minima and PAC-Bayes

An older line of work connects generalization to the **flatness** of the minimum: a minimum in a flat region of the loss surface generalizes better than a minimum in a sharp region, because small perturbations to the parameters do not change the loss. The intuition was formalized by [hochreiter1997flat] and given a PAC-Bayesian foundation by [dziugaite2017computing].

The **PAC-Bayes bound** states that, with high probability over the sample,

$$\mathcal{L}(\theta) \leq \hat{\mathcal{L}}_Q(\theta) + \sqrt{\frac{D_{\text{KL}}(Q\|P) + \log(2\sqrt{N}/\delta)}{N}},$$

where Q is a “posterior” over parameters (typically a Gaussian centered at the trained weights), P is a “prior”, and $\hat{\mathcal{L}}_Q$ is the empirical loss under Q . The bound is non-vacuous when the posterior is concentrated near the trained weights (small KL) and the empirical loss is low.

PAC-Bayes bounds have been made non-vacuous for small networks ([dziugaite2017computing]), but extending them to modern large networks remains an open problem. The principal obstacle is the KL term: for a network

with 10^9 parameters, even a Gaussian posterior with variance 10^{-6} has a KL of order 10^3 , which is too large.

16.4 Compression-based bounds

A separate line of work bounds generalization via **compression**: if the trained network can be compressed to k bits with no loss in training accuracy, then the generalization gap is $O(\sqrt{k/N})$, regardless of the original parameter count.

[arorazo18stronger] proved compression bounds for specific architectures; [zhou2018compressing] and others have shown that trained networks can be heavily compressed (often by $10\times$ – $100\times$) with minimal loss. The compression bound is appealing because it connects to the empirical practice of pruning and quantization. It is, however, hard to make tight: the achievable compression depends on the network and the training procedure, and the bound is only as good as the compression.

16.5 The neural tangent kernel

For wide networks at initialization, the dynamics of gradient descent simplify dramatically. [jacot2018neural] showed that, in the infinite-width limit, the function $f_\theta(\mathbf{x})$ evolves under gradient descent according to a kernel regression with the **neural tangent kernel** (NTK):

$$K(\mathbf{x}, \mathbf{y}) = \mathbb{E}_{\theta \sim \text{init}} [\langle \nabla_{\theta} f_{\theta}(\mathbf{x}), \nabla_{\theta} f_{\theta}(\mathbf{y}) \rangle].$$

The optimization in function space is convex, even though the optimization in parameter space is not. Generalization in the NTK regime can be analyzed with kernel-method theory, giving rates that match the empirical scaling laws in some regimes.

The NTK has substantial explanatory power in the infinite-width limit. Its limitations are equally substantial: real networks are not infinitely wide, and they do not stay in the NTK regime during training (the kernel changes substantially). The NTK is best understood as a *linearization* of the training dynamics around initialization; it captures the early-time dynamics and the wide-network limit, but not the feature-learning dynamics that distinguish deep learning from kernel methods.

16.6 The lottery ticket hypothesis

[frankle2018lottery] proposed the **lottery ticket hypothesis**: a randomly initialized dense network contains a sparse subnetwork (“the winning ticket”) that, when trained in isolation from the original initialization, matches the test accuracy of the dense network. The hypothesis has substantial empirical support — winning tickets can be pruned by 90% or more without loss — and provides a natural connection to compression-based generalization bounds.

The lottery ticket hypothesis is an empirical observation, not a theorem. Theoretical follow-ups ([malach2020proving]) have shown that sufficiently overparameterized networks contain winning tickets with high probability, providing a partial theoretical foundation.

16.7 The state of the theory

The honest state of deep learning generalization theory in 2025 is that **no single explanation is fully satisfactory**. The classical theory is vacuous. The implicit bias program is illuminating in restricted regimes but does not extend cleanly to realistic networks. PAC-Bayes is non-vacuous only for small networks. Compression bounds are appealing but hard to make tight. The NTK captures only the linearized dynamics. The lottery ticket hypothesis is empirical.

What the field has converged on is a **constellation** of mechanisms: implicit bias selects simple solutions among the many that fit; flatness and compression are correlated with generalization; wide networks are analyzable via the NTK; sparsity and pruning preserve performance. A unified theory that integrates these mechanisms is the principal open problem of the field (see Chapter 15).

16.8 Computational experiments

16.8.1 Experiment 1: Double descent

Train a network of varying width on a fixed dataset. Plot training and test error as a function of width. Verify the double descent phenomenon.

16.8.2 Experiment 2: Random labels

Train a network on (a) true labels and (b) random labels. Compare training and test accuracy. Verify [zhang2021understanding]’s observation.

16.8.3 Experiment 3: Lottery ticket

Train a network, prune by 90%, retrain the pruned subnetwork from (a) the original initialization and (b) a new initialization. Compare test accuracy.

16.9 Historical notes

The classical learning theory is due to [vapnik1971uniform] (VC dimension) and [bartlett2002rademacher] (Rademacher complexity). The breakdown of classical theory in deep learning was emphasized by [zhang2021understanding]. Double descent was systematically studied by [belkin2019reconciling]. Implicit bias of gradient descent in linear models is due to [soudry2018implicit] and [ji2019implicit]; the deep-network case is treated in [neyshabur2014search]. PAC-Bayes bounds are due to [mcallester1999pac]; modern non-vacuous bounds to [dziugaite2017computing]. The NTK is due to [jacot2018neural]. The lottery ticket hypothesis is due to [frankle2018lottery].

16.10 Exercises

1. **(Easy)** Compute the classical uniform-convergence bound for a ResNet-50 ($P \approx 25\text{M}$) trained on ImageNet ($N \approx 1.3\text{M}$). Verify that it is vacuous.
2. **(Easy)** State the lottery ticket hypothesis. What is the role of the original initialization?
3. **(Medium)** Reproduce the [zhang2021understanding] experiment on a small dataset: train a network on true labels and on random labels. Compare training and test accuracy.
4. **(Medium)** Reproduce the double descent phenomenon: train a network of varying width on a fixed dataset. Plot test error as a function of width.
5. **(Medium)** Reproduce the lottery ticket experiment: train a network, prune by 90%, retrain from the original initialization and from a fresh initialization. Compare.
6. **(Hard)** Implement the PAC-Bayes bound of [dziugaite2017computing] for a small network. Make the bound non-vacuous.
7. **(Hard)** Implement the NTK for a small MLP. Compare its predictions to the actual training dynamics.
8. **(Research)** Investigate the relationship between implicit bias and architecture. How does the implicit bias of a CNN differ from that of an MLP? From that of a transformer?

Interpretability

Mechanistic interpretability is the program of reverse-engineering trained neural networks: understanding, in human-meaningful terms, the computations the network performs. The program is motivated by safety (we cannot align systems we do not understand), by science (deep networks are among the most complex artifacts humans have built, and their internal computations are largely opaque), and by engineering (understanding failure modes is the first step to fixing them).

This chapter surveys the principal approaches: **feature visualization** (what does a neuron represent?), **circuits** (how do neurons compose into computations?), **sparse autoencoders** (decomposing polysemantic neurons into monosemantic features), and **causal interventions** (testing hypotheses about computational structure). The chapter treats each method’s strengths, limitations, and current frontiers.

17.1 Feature visualization

The simplest interpretability question is: *what does a neuron represent?* **Feature visualization** [olah2017feature] answers this question by optimization: find the input that maximally activates a given unit. The optimization is regularized to produce natural-looking images, since the unconstrained optimum is often an adversarial-like pattern.

Feature visualization is informative for individual neurons in vision models but has limitations: (i) the most-activating natural images often tell a different story than the optimized inputs; (ii) polysemantic neurons (Chapter 11) have no single concept to visualize; (iii) the method is most useful for low-level features and less useful for high-level abstractions.

17.2 Circuits

A **circuit** is a subnetwork of a trained network that implements a specific computation. [olah2020zoom] identified circuits in vision transformers: curve detectors, orientation detectors, color opponency circuits, and others. The methodology combines feature visualization (to identify what each unit represents), weight inspection (to identify which units connect to which), and ablation (to test

whether the connections are causally responsible for the computation).

The circuits program has been productive for early layers of vision models, where computations are relatively simple and the units are often monosemantic. For later layers and for language models, the situation is harder: polysemanticity is widespread, the relevant computations are more abstract, and circuits span more of the network.

The current frontier is **mechanistic interpretability of language models**: identifying the circuits by which transformers implement specific syntactic computations (subject-verb agreement, indirect object identification, in-context learning). [wang2022interpretability] identified the circuit for indirect object identification in GPT-2 small; [olson2022incontext] identified “induction heads” as a mechanism for in-context learning. These case studies are encouraging but cover only a small fraction of the computations performed by the models.

17.3 Sparse autoencoders for interpretability

As introduced in Chapter 11, sparse autoencoders (SAEs) decompose polysemantic neurons into monosemantic features. The methodology has become a principal tool of mechanistic interpretability, allowing researchers to identify the features that the network actually uses (as opposed to the neurons it happens to have).

The state of the art is rapidly evolving. [bricken2023monosemanticity] trained SAEs on a small transformer and recovered interpretable features; [templeton2024scaling] scaled this to Claude 3 Sonnet and recovered features corresponding to a wide range of concepts (countries, code, biases, intentions). The features are causally meaningful: activating or suppressing a specific feature changes the model’s behavior in predictable ways.

17.4 Causal interventions

The strongest form of interpretability evidence is **causal**: not just “this unit represents X” but “this unit is *used* by the network to compute X.” Causal

interventions test the latter by perturbing the unit and observing the effect.

Activation patching (Chapter 11) replaces activations from one input with those from another; the effect on the output identifies the causal role. **Path patching** does the same for specific paths through the network. **Ablation** zeros out units and measures the effect. **Activation steering** scales a specific direction up or down, allowing controlled manipulation of the model’s behavior.

The combination of SAEs (for identifying features) and causal interventions (for testing their roles) is the current methodological frontier. The principal open question is **scalability**: methods that work on small models (GPT-2 small, 124M parameters) are expensive to apply to large models (GPT-4-class, ~1T parameters). Whether interpretability can scale with model size is one of the central open problems of the field.

17.5 The microscope vs. MRI distinction

[olah2022theoretical] distinguishes two complementary modes of interpretability research:

- **Microscope**: zoom in on a specific computation, identify the circuit, test the mechanism. High-resolution, low-coverage.
- **MRI**: scan the whole model for patterns, identify common motifs, build a topographic map. Low-resolution, high-coverage.

Most current interpretability research is microscope-style: detailed studies of specific circuits in specific models. The MRI mode — systematic surveys of entire models — is methodologically harder and less developed, but is necessary if interpretability is to scale.

17.6 Computational experiments

17.6.1 Experiment 1: Feature visualization

Implement feature visualization for a pretrained CNN. Optimize inputs to maximally activate specific neurons. Compare the optimized inputs to the top natural images for the same neurons.

17.6.2 Experiment 2: Circuit identification

Identify a simple circuit (e.g., curve detector) in the early layers of a pretrained vision model. Verify the circuit by ablation: zero out the circuit’s units and measure the effect on the relevant behavior.

17.6.3 Experiment 3: Activation steering

Train a small SAE on a pretrained language model’s activations. Find a feature corresponding to a behavioral property (e.g., refusal, sycophancy). Steer the model by scaling the feature’s activation up or down; verify the behavioral change.

17.7 Historical notes

Feature visualization has roots in [simonyan2013deep]; the modern form is due to [olah2017feature]. The circuits program was launched by [olah2020zoom]. The sparse-autoencoder methodology for interpretability is due to [bricken2023monosemanticity] and [templeton2024scaling]. Activation patching as an interpretability tool was introduced by [geiger2021causal] and [vig2020investigating]. The induction-head discovery is due to [olson2022incontext]. The microscope/MRI distinction is due to [olah2022theoretical].

17.8 Exercises

1. **(Easy)** Implement feature visualization for a single neuron in a pretrained CNN. Compare the optimized input to the top natural images for the same neuron.
2. **(Easy)** Implement a simple ablation: zero out a specific unit in a pretrained model and measure the effect on a specific behavior.
3. **(Medium)** Implement activation patching on a small transformer. Use it to identify the positions causally responsible for a specific output token.
4. **(Medium)** Train a small SAE on a pretrained model’s activations. Inspect the learned features. Are they monosemantic?
5. **(Medium)** Implement path patching. Use it to identify the circuit by which a small transformer solves a simple algorithmic task.

6. **(Hard)** Implement activation steering. Find a direction in a language model's activations corresponding to a behavioral property, and steer the model by scaling the direction.
7. **(Hard)** Reproduce the induction-head experiment of [olsson2022incontext] in a small transformer.
8. **(Research)** Investigate the scalability of interpretability methods. How do the cost and the yield of SAE-based interpretability scale with model size?

Open Problems

This chapter enumerates twenty open problems that we believe will shape the next decade of deep learning research. The list is necessarily partial and reflects our own biases; we have favored problems that are *precisely stateable*, *empirically motivated*, and *not obviously impossible*. For each, we give the current state of knowledge, the principal obstacles, and pointers to the relevant literature. The problems are organized by area, but many overlap.

We do not claim this list is comprehensive. A reader looking for a different emphasis should consult the open-problem lists of [bengio2014iclr], [goodfellow2016deep] (Chapter 5), and the various *Frontiers in AI* reports.

18.1 Optimization

18.1.1 Problem 1: A satisfactory convergence theory for Adam

The Adam optimizer is universally used and universally under-theorized. The [kingma2014adam] convergence proof requires a decreasing step size, which is not used in practice; the [reddi2018convergence] counterexample shows that constant-step Adam can diverge on simple convex problems; AMSGrad fixes this but is not used in practice. A convergence theory that matches practice is the principal open problem of optimizer theory.

The obstacle is that Adam’s behavior depends on the interaction between the momentum terms, the second-moment scaling, and the geometry of the loss surface, and these interactions are difficult to analyze cleanly. A candidate approach is the *biased-gradient* framework of [bottou2018optimization], but extending it to Adam’s adaptive scaling is non-trivial.

18.1.2 Problem 2: Why does SGD find good minima in non-convex landscapes?

The empirical fact that SGD finds good minima in deep networks’ highly non-convex loss landscapes is the foundational puzzle of optimization in deep learning. The candidate explanations — implicit bias, landscape connectivity, the spin-glass picture — are partial (Chapter 2). A unified theory is missing.

The obstacle is that the relevant property is not about a single optimization trajectory but about the *distribution* of trajectories that SGD visits, and this distribution depends on the interaction between the geometry of the loss surface, the noise structure of mini-batch gradients, and the discrete-time dynamics of SGD.

18.1.3 Problem 3: The mechanism of scaling laws

The empirical scaling laws (Chapter 12) are remarkably robust but theoretically unexplained. Why does the loss scale as a power law in compute, with exponents that are remarkably consistent across architectures? The candidate explanations — statistical mechanics, information theory, random features — capture aspects but not the whole.

The obstacle is that the scaling law is a statement about the *learning dynamics* of the network, not just the function class, and the dynamics are hard to analyze.

18.2 Generalization

18.2.1 Problem 4: A unified theory of generalization in deep learning

The classical theory is vacuous; the modern candidates (implicit bias, PAC-Bayes, compression, NTK, lottery ticket) are partial (Chapter 13). A unified theory that integrates these mechanisms and gives non-vacuous bounds for realistic networks is the central open problem of generalization theory.

The obstacle is that the mechanisms interact: implicit bias selects solutions with low complexity, but the relevant complexity measure depends on the architecture and the optimizer; compression bounds work but require explicit compression; PAC-Bayes is non-vacuous only for small networks. A theory that integrates these mechanisms requires understanding their interaction.

18.2.2 Problem 5: Double descent beyond the simple regimes

Double descent has been documented in many settings, but the precise conditions under which it occurs — and the precise mechanisms by which it oc-

curs — are not fully understood. When does the second descent begin? Why does it eventually saturate? Does it occur in language models at scale?

The obstacle is that double descent requires tracking both the optimization trajectory and the test loss, and the two are coupled in non-trivial ways.

18.2.3 Problem 6: Generalization under distribution shift

Real-world deployment involves distribution shift: the test distribution differs from the training distribution. The current theory of distribution shift — domain adaptation, robust optimization, invariance — is partial. A predictive theory of when and how networks generalize under distribution shift is missing.

The obstacle is that distribution shift is heterogeneous: it can be a covariate shift, a label shift, a concept drift, or a more complex structural change, and each requires different theoretical tools.

18.3 Representation

18.3.1 Problem 7: A theory of feature learning

Deep networks learn features, but the feature-learning process is poorly understood. The NTK captures only the linearized dynamics; feature-learning dynamics require going beyond the NTK. The *mean-field* and *feature-learning* limits of [yang2021tensor] and [roberts2022principles] are partial steps.

The obstacle is that feature learning involves *changes* in the kernel during training, and analyzing these changes requires tools that go beyond the linearization.

18.3.2 Problem 8: The geometry of learned representations

Why are representations often linear? Why are they sometimes in superposition (Chapter 11)? A predictive theory of the geometry of learned representations is missing.

The obstacle is that the geometry depends on the interaction between the architecture, the data, and the optimizer, and the interaction is hard to analyze.

18.3.3 Problem 9: Disentanglement

The goal of disentanglement — learning representations in which each semantically meaningful factor corresponds to a single dimension — has been pursued since the early days of representation learning. Despite substantial work, no method reliably achieves disentanglement without supervision. Theoretical results ([locatello2019challenging]) show that unsupervised disentanglement is fundamentally ill-posed without inductive bias.

The obstacle is that the “true” factors of variation are not uniquely defined by the data distribution; they depend on the downstream task. A theory that integrates the task into the disentanglement criterion is missing.

18.4 Architectures

18.4.1 Problem 10: Beyond the transformer

The transformer has dominated for seven years. Is there a better architecture, and if so, what is its inductive bias? Candidate alternatives — Mamba [gu2023mamba], RWKV [peng2023rwkv], Hyena [poliz2023hyena] — are competitive in some regimes but have not displaced the transformer.

The obstacle is that the transformer’s success is partly due to its hardware friendliness (parallelism, memory access patterns), and any alternative must compete on hardware as well as on mathematical structure.

18.4.2 Problem 11: The role of recurrence in long-context models

Transformers have $O(n^2)$ attention cost; recurrent models have $O(n)$ but a more limited function class. The right trade-off for long-context modeling — the architecture that combines the expressivity of attention with the linear scaling of recurrence — is open.

18.4.3 Problem 12: Modular and compositional architectures

Biological brains are modular; current deep networks are largely monolithic. A theory of *modular composition* — when should the network be split into modules, how should the modules communicate,

how should the modules be specialized — is missing. Mixture-of-experts [shazeer2017outrageously] is a partial step.

18.5 Interpretability

18.5.1 Problem 13: Scalable mechanistic interpretability

Current interpretability methods work on small models. Scaling them to models with 10^{11} – 10^{13} parameters is open (Chapter 14). The obstacle is that the methods are labor-intensive: each circuit must be identified and validated by hand.

18.5.2 Problem 14: Automated circuit discovery

Manual circuit discovery does not scale. Automated methods — sparse autoencoders, dictionary learning, attribution — are promising but immature. A method that automatically identifies the circuits in a large network is the principal methodological open problem of interpretability.

18.5.3 Problem 15: The relationship between interpretability and alignment

The intended use of interpretability is alignment: ensuring that networks do what we want, not what they appear to do. The connection between interpretability and alignment is, however, not fully worked out. Does interpretability actually help alignment, or does it just give us a false sense of understanding?

18.6 Training and data

18.6.1 Problem 16: The data wall

High-quality text data is finite and may be exhausted at scales of 10^{13} – 10^{14} tokens (Chapter 12). What happens then? Candidate solutions — synthetic data, data replay, multimodal data — are partial. A theory of the *data requirements* of large models, beyond the Chinchilla law, is open.

18.6.2 Problem 17: Curriculum and data ordering

The order in which data is presented affects the final model, but the effect is poorly understood. Optimal curricula are not known; the heuristics in use (easy-to-hard, balanced sampling) have only weak theoretical justification.

18.6.3 Problem 18: The role of model and data quality

Not all data is equally useful; not all parameters are equally informative. A theory that integrates *quality* into the scaling laws — predicting how the loss scales with data quality, or how the optimal allocation changes with model architecture — is open.

18.7 Theory and practice

18.7.1 Problem 19: A theory of in-context learning

Large language models learn from context: a few examples in the prompt improve performance on a new task. The mechanism by which in-context learning works — and the conditions under which it works — are only partially understood. The induction-head discovery ([olsson2022incontext]) is a step; a complete theory is open.

18.7.2 Problem 20: The relationship between deep learning and statistics

Deep learning is, in some sense, a statistical method, but the relationship to classical statistics is murky. Many classical statistical concepts (consistency, efficiency, sufficiency, the Cramer-Rao bound) have analogs in deep learning, but the analogs are partial. A *statistical theory of deep learning* that integrates the classical concepts with the empirical phenomena — a theory that explains why deep networks are surprisingly good statistical estimators despite their huge parameter counts — is the grand open problem of the field.

18.8 Coda

The twenty problems above are not the only open problems in deep learning. They are a partial list,

biased by our own interests and by the limits of our knowledge. We offer them in the spirit of [hilbert1900mathematical]: as a guide to the principal work of the next decade, and as an invitation to the reader to take up the problems we have not been able to solve.

Deep learning is, in 2025, a field with enormous empirical success, substantial theoretical progress, and large open theoretical questions. The next decade will, we hope, see the open problems closed, the empirical record systematized, and the theory brought into line with practice. We hope this monograph contributes to that work.

BACK MATTER

References

The monograph cites approximately 167 works from the deep learning, mathematics, and machine learning literature. The complete BibTeX bibliography is in `references.bib`.

Below is a selection of the most-cited primary sources, organized by topic.

9 Foundational deep learning

- **LeCun, Bengio, Hinton (2015).** *Deep learning*. Nature. `lecun2015deep`
- **Goodfellow, Bengio, Courville (2016).** *Deep Learning*. MIT Press. `goodfellow2016deep`
- **Bengio (2009).** *Learning deep architectures for AI*. Foundations and Trends in ML. `bengio2009learning`
- **Rumelhart, Hinton, Williams (1986).** *Learning representations by back-propagating errors*. Nature. `rumelhart1986learning`

10 Transformers and attention

- **Vaswani et al. (2017).** *Attention is all you need*. NeurIPS. `vaswani2017attention`
- **Devlin et al. (2019).** *BERT: Pre-training of deep bidirectional transformers*. NAACL. `devlin2019bert`
- **Brown et al. (2020).** *Language models are few-shot learners*. NeurIPS. `brown2020language`
- **Bahdanau, Cho, Bengio (2015).** *Neural machine translation by jointly learning to align and translate*. ICLR. `bahdanau2015neural`

11 Optimization

- **Kingma & Ba (2015).** *Adam: A method for stochastic optimization*. ICLR. `kingma2014adam`
- **Loshchilov & Hutter (2019).** *Decoupled weight decay regularization (AdamW)*. ICLR. `loshchilov2019decoupled`
- **Nesterov (1983).** *A method for solving the convex programming problem with convergence rate $O(1/k^2)$* . Doklady. `nesterov1983method`
- **Robbins & Monro (1951).** *A stochastic approximation method*. Annals of Math. Stat. `robbins1951stochastic`
- **Reddi, Kale, Kumar (2018).** *On the convergence of Adam and beyond*. ICLR. `reddi2018convergence`

12 Architectures

- **Krizhevsky, Sutskever, Hinton (2012).** *ImageNet classification with deep CNNs*. NeurIPS. `krizhevsky2012imagenet`
- **He et al. (2016).** *Deep residual learning for image recognition*. CVPR. `he2015deep`
- **Hochreiter & Schmidhuber (1997).** *Long short-term memory*. Neural Computation. `hochreiter1997long`
- **LeCun et al. (1998).** *Gradient-based learning applied to document recognition*. Proc. IEEE. `lecun1998gradient`

13 Generative models

- **Kingma & Welling (2014).** *Auto-encoding variational Bayes*. ICLR. `kingma2013auto`
- **Goodfellow et al. (2014).** *Generative adversarial nets*. NeurIPS. `NIPS2014_5dce8a06`
- **Ho, Jain, Abbeel (2020).** *Denoising diffusion probabilistic models*. NeurIPS. `ho2020denoising`
- **Song et al. (2021).** *Score-based generative modeling through SDEs*. ICLR. `song2021scorebased`

14 Generalization theory

- **Zhang, Bengio, Hardt, Recht, Vinyals (2021).** *Understanding deep learning requires rethinking generalization*. Comm. ACM. `zhang2021understanding`
- **Belkin, Hsu, Ma, Mandal (2019).** *Reconciling modern ML practice and the bias-variance trade-off*. PNAS. `belkin2019reconciling`
- **Jacot, Gabriel, Hongler (2018).** *Neural tangent kernel: convergence and generalization*. NeurIPS. `jacot2018neural`
- **Frankle & Carbin (2019).** *The lottery ticket hypothesis*. ICLR. `frankle2018lottery`

15 Scaling laws

- **Kaplan et al. (2020).** *Scaling laws for neural language models*. arXiv. `kaplan2020scaling`
- **Hoffmann et al. (2022).** *Training compute-optimal large language models (Chinchilla)*. arXiv. `hoffmann2022training`

16 Interpretability

- Olah et al. (2020). *Zoom in: an introduction to circuits*. Distill. olah2020zoom
- Elhage et al. (2022). *Toy models of superposition*. Transformer Circuits Thread. elhage2022superposition
- Templeton et al. (2024). *Scaling monosemanticity*. Anthropic. templeton2024scaling
- Katharopoulos et al. (2020). *Transformers are RNNs: linear attention*. ICML. katharopoulos2020linear

For the complete bibliography with 167 entries, see `references.bib`.

17 Mathematical foundations

- Strang (2016). *Introduction to Linear Algebra*. Wellesley-Cambridge. strang2016introduction
- Boyd & Vandenberghe (2004). *Convex Optimization*. Cambridge. boyd2004convex
- Cover & Thomas (1999). *Elements of Information Theory*. Wiley. cover1999elements
- Nesterov (2018). *Lectures on Convex Optimization*. Springer. nesterov2018lectures

18 Information theory in deep learning

- Tishby & Zaslavsky (2015). *Deep learning and the information bottleneck principle*. ITW. tishby2015deep
- Shannon (1948). *A mathematical theory of communication*. Bell System Tech. J. shannon1948mathematical

19 Training practice

- Ioffe & Szegedy (2015). *Batch normalization*. ICML. ioffe2015batch
- Ba, Kiros, Hinton (2016). *Layer normalization*. arXiv. ba2016layer
- Srivastava et al. (2014). *Dropout*. JMLR. srivastava2014dropout
- He et al. (2015). *Delving deep into rectifiers (He initialization)*. ICCV. he2015delving
- Glorot & Bengio (2010). *Understanding the difficulty of training deep feedforward networks*. AIS-TATS. glorot2010understanding

20 Efficient attention

- Dao et al. (2022). *FlashAttention*. NeurIPS. dao2022flashattention